

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук

Кафедра программирования и информационных технологий

Разработка мобильного приложения для перерисовки фотографий под одну
из заранее определенных стилистик «Visionary»

Курсовая работа

09.03.02 Информационные системы и технологии

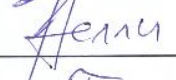
Программная инженерия в информационных системах

Зав. кафедрой _____ д.ф.-м.н., доцент, С.Д. Махортов

Руководитель _____ ст. преподаватель, В.С. Тарасов

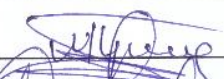
Руководитель практики _____ ассистент, Ю.В. Шишко

Обучающийся  _____ Д.В. Кретов, 3 курс, д/о

Обучающийся  _____ Н.В. Агапова, 3 курс, д/о

Обучающийся  _____ А.Е. Губанов, 3 курс, д/о

Обучающийся  _____ Д.И. Ханин, 3 курс, д/о

Обучающийся  _____ М.А. Прилепин, 3 курс, д/о

Обучающийся  _____ Д.А. Набокин, 3 курс, д/о

Воронеж 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 Постановка задачи.....	7
1.1 Цели создания приложения.....	7
1.2 Задачи приложения	7
1.3 Требования к приложению.....	8
1.3.1 Требования к приложению в целом	8
1.3.2 Требования к функциям (задачам), выполняемым приложением	8
1.3.3 Требования к оформлению и верстке страниц.....	9
1.3.4 Требования к защите информации	9
1.4 Задачи, решенные в процессе разработки	9
2 Анализ предметной области	11
2.1 Глоссарий	11
2.2 Обзор аналогов	12
2.2.1 PicsArt	12
2.2.2 DeepArt	13
2.2.3 Вывод по обзору аналогов	13
3 Моделирование системы	15
3.1 Диаграмма прецедентов (Use Case).....	15
3.2 Диаграмма состояний (Statechart diagram)	15
3.3 Диаграмма активностей.....	16
3.4 ER-диаграмма	18
3.5 Функциональная схема	18
3.6 Диаграмма развертывания (Deployment diagram)	20
4 Реализация.....	22
4.1 Средства реализации.....	22
4.2 Архитектура клиентской части.....	24
4.2.1 Компонент представления (View)	25
4.2.2 Компонент данных (Model).....	26
4.2.3 Компонент модели представления (ViewModel)	28
4.2.4 Слой utils	29
4.3 Архитектура серверной части.....	29

4.3.1 Структура проекта.....	30
4.4 Реализация функциональных требований приложения	33
4.4.1 Функции для пользователей.....	33
4.4.2 Функции для авторизированных пользователей	33
4.4.3 Функции для администратора.....	35
4.4.4 Требования к работе приложения	36
4.5 Нефункциональные требования	37
4.5.1 Безопасность	37
4.5.2 Производительность	38
4.5.3 Совместимость	39
4.5.4 Гибкость	39
4.6 Реализация интерфейса приложения	40
4.6.1 Загрузочный экран	40
4.6.2 Начальный экран	41
4.6.3 Экран входа.....	42
4.6.4 Экран регистрации	43
4.6.5 Страница ленты с постами	44
4.6.6 Страницы для обработки изображений	46
4.6.7 Страница личного кабинета	53
4.6.8 Страница настроек	55
4.7 Реализация интерфейса админ-панели.....	57
4.7.1 Главная страница админ панели.....	58
4.7.2 Работа с базой данных PostgreSQL	58
4.7.3 Просмотр документации по API приложения.....	60
4.7.4 Страница доступа к базе MinIO	61
4.7.5 Просмотр метрик при помощи Grafana	62
4.7.6 Просмотр метрик и предупреждений в Prometheus.....	62
4.7.7 Просмотр и управление очередями сообщений в RabbitMQ.....	63
5 Тестирование	63
6 Аналитика	65
6.1 Воронки	65
6.2 Статистика	67

ЗАКЛЮЧЕНИЕ	69
------------------	----

ВВЕДЕНИЕ

В современном мире визуальный контент становится неотъемлемой частью общения и самовыражения. Фотографии и изображения сопровождают нас повсюду — в социальных сетях, в рекламных материалах, в личных блогах и профессиональных портфолио. Возможность преобразовать обычное фото в художественную картину, подчеркивающую индивидуальность автора, обогащает досуг и стимулирует творческое мышление.

С появлением нейросетевых методов переноса стиля любому пользователю стал доступен мощный инструмент цифрового творчества: достаточно загрузить снимок и выбрать желаемый художественный стиль и уже через несколько секунд изображение преобразуется. Однако большинство существующих сервисов либо требуют доступа к десктопному ПО, либо требуют больших денежных затрат со стороны пользователя за использование, что сдерживает их широкое распространение.

В ходе анализа аналогов (DeepArt, PicsArt) выявлено, что пользователю часто приходится долго разбираться в интерфейсе, чтобы освоить базовый функционал, терпеть задержки при обработке или мириться с ограниченным числом стилей. Кроме того, отсутствие единой адаптированной под смартфон платформы усложняет хранение и обмен готовыми работами.

Исходя из этих наблюдений и опираясь на техническое задание «Visionary» — мобильного приложения для перерисовки фотографий и картинок в выбранный стиль при помощи нейронных сетей — мы разработали решение, объединяющее быструю обработку, минималистичный интерфейс и возможность кастомизации художественных эффектов.

Актуальность обусловлена необходимостью автоматизации процесса художественной стилизации изображений, которая повышает доступность цифрового творчества и удовлетворяет растущий спрос на персонализированный визуальный контент.

Целью курсовой работы является разработка мобильного приложения, обеспечивающего эффективную и интерактивную стилизацию фото с помощью нейросетевых алгоритмов.

Задачи:

- Изучить теоретические основы переноса стиля и современные нейронные архитектуры;

- Провести обзор существующих аналогов и определить их сильные и слабые стороны;

- Спроектировать архитектуру мобильного приложения (клиент, сервер, модель нейросети);

- Реализовать ключевые модули обработки изображений и пользовательский интерфейс.

1 Постановка задачи

1.1 Цели создания приложения

Целями создания мобильного приложения Visionary являются:

— создание сервиса по обработке изображений под некоторый набор стилистик при помощи нейросети, что позволит пользователю получать уникальные изображения;

— осуществление системы по опубликованию изображений как внутри приложения для обмена результатами работы и возможностями оценивания, так и экспорта в другие источники, что обеспечит рекламу приложению и привлечёт новых пользователей;

1.2 Задачи приложения

Приложение позволяет решать следующие задачи:

- просматривать опубликованные в приложении посты;
- реагировать на посты в ленте — оставлять лайки и комментарии;
- просматривать статистику своего профиля — количество лайков, подписчиков и подписок;
- использовать свои фотографии для стилизации в приложении;
- изменять общий стиль изображения при помощи предустановленных наборов стилистик;
- экспортировать обработанные изображения в сторонние приложения или хранилище устройства;
- применять фильтры к фотографиям перед или после стилизации (яркость, контрастность);
- управлять данными личного профиля (никнейм, пароль);
- сохранять изображения на сервере.

1.3 Требования к приложению

1.3.1 Требования к приложению в целом

Данное приложение должно удовлетворять следующим основным требованиям:

- приложение должно корректно работать на устройствах, работающих на операционной системе Android 8.0 и новее;
- реализовывать все поставленные задачи.

1.3.2 Требования к функциям (задачам), выполняемым приложением

Приложение обеспечивает пользователю выполнение следующих функций:

Для всех пользователей:

- авторизация и регистрация (через почту);
- просмотр ленты постов с возможностью сортировки и фильтрации;
- открытие постов, чтение комментариев;
- сохранение и экспорт фотографий из постов;
- просмотр профилей других пользователей;
- поиск авторов по имени.

Для авторизованных пользователей:

- доступ к личному профилю и его редактирование (имя, пароль);
- переключение между вкладками «личное» и «публичное»;
- публикация обработанных изображений;
- использование встроенного редактора с стилями;
- загрузка изображений из галереи или камеры;
- установка приватности поста (личное/публичное);
- взаимодействие с чужим и своим контентом (лайки, комментарии);

- выход из аккаунта и удаление профиля.

Функции для пользователей с подпиской:

- применение платных стилей обработки;
- увеличение недельного лимита на количество обработок.

1.3.3 Требования к оформлению и верстке страниц

Оформление и верстка интерфейса удовлетворяет следующим требованиям:

- приложение оформлено в едином стиле с разработанным названием;
- использована ограниченная, контрастная цветовая палитра и единый набор шрифтов;
- корректное и одинаковое отображение на экранах от 5 до 7 дюймов.

1.3.4 Требования к защите информации

Для обеспечения безопасности информации применяется стандарт для безопасного обмена данными JWT. Даже если злоумышленник получит токен, который предоставляет доступ ко всем функциям приложения, он станет недействительным через 15 минут и злоумышленнику нужно будет получить новый. Также используется механизм защиты от SQL-инъекций и шифрование персональных данных пользователей.

1.4 Задачи, решенные в процессе разработки

Были поставлены и выполнены следующие задачи:

- анализ предметной области;
- обзор аналогов;
- постановка задачи;
- создание репозитория GitHub и доски в Jira;

- разработка требований: к приложению в общем, к функциям, к структуре, к программному обеспечению, к оформлению и верстке страниц, к защите информации;
- создание диаграмм: use-case, состояний, активностей, функциональная схема, развёртывания, ER;
- разработка дизайна приложения;
- написание технического задания в соответствии с ГОСТ 34.602 – 2020;
- реализация моделей нейросети для обработки изображений;
- реализация интерфейса приложения;
- реализация админ-панели;
- реализация серверной части приложения;
- развертывание приложения;
- написание курсовой работы.

2 Анализ предметной области

2.1 Глоссарий

В настоящей работе используются следующие термины и сокращения с соответствующими определениями:

— Frontend – это клиентская часть продукта (интерфейс, с которым взаимодействует пользователь);

— Backend – это логика и функциональность, которая реализуется на сервере для обеспечения работы приложения;

— Серверная часть – это инфраструктура, на которой работает backend, включая оборудование, операционные системы, сети и системы хранения данных.;

— Сервер – это устройство, в частности компьютер, которое отвечает за предоставление услуг, программ и данных другим клиентам посредством использования сети;

— Нейросеть – это программа, которая повторяет модель человеческих нейронных связей и на их основе создаёт программу, которая умеет распознавать и генерировать контент (картинки, видео, текст и так далее);

— Фильтр – это операция, имеющая своим результатом изменения характеристик и параметров изображения, получаемые из исходного по некоторым правилам;

— Лайк – это обозначение, что другому пользователю понравилась ваша публикация;

— Подписчик – это человек, который подписался на ваши обновления, то есть выразил заинтересованность результатами вашего труда;

— Деплой – это размещение готовой версии программного обеспечения на платформе, доступной для пользователей;

— Контент – это результат работы, в виде постов, пользователей приложения;

— Пост — это совокупность изображения, имени автора и комментариями (при их наличии), рассматривающееся как целое;

— Публикация — это альтернативное обозначение поста;

— Подписки — это список пользователей, на чьи обновления подписался конкретный пользователь, то есть выразил заинтересованность в их контенте или деятельности;

— Лента – это обозначение визуальной оболочки для просмотра общедоступных постов;

— Сайт — это графическая оболочка для взаимодействия с пользователями, представляющая собой совокупность веб-страниц, объединенных общей структурой и дизайном;

— Аватарка — графическое представление пользователя, его альтернативная личность в приложении.

2.2 Обзор аналогов

2.2.1 PicsArt

PicsArt — мощный универсальный фоторедактор с широким набором инструментов: AI-фильтры, коллажи, монтаж и интеграция с социальными сетями. Приложение доступно на Android и iOS, имеет множество дополнительных функций и обширную библиотеку эффектов.

Достоинства:

- многофункциональность и глубокая интеграция с соцсетями;
- обширная библиотека фильтров и шаблонов;
- постоянное расширение AI-возможностей и партнёрства с брендами.

Недостатки:

- перегруженный интерфейс, сложный для новичков;

- высокая стоимость подписки (от \$13/мес.) затрудняет массовое распространение среди аудитории;

- конкуренция с профессиональными решениями (Adobe), предлагающими более профессиональные инструменты.

2.2.2 DeepArt

DeepArt специализируется на переносе стиля известных художников (Ван Гог, Пикассо) и создаёт высокохудожественные изображения. Приложение фокусируется на качестве конечного результата и эстетических эффектах.

Достоинства:

- уникальные арт-эффекты и высокое качество стилизации;
- возможности партнёрств с художниками и расширение форматов (видео).

Недостатки:

- медленная скорость обработки (3–5 минут), что может отпугнуть пользователей;
- узкая специализация, отсутствие широкого набора инструментов для базового редактирования;
- рост конкуренции в сегменте AI-арт.

2.2.3 Вывод по обзору аналогов

Сравнительный анализ показывает, что ни один из конкурентов не сочетает в себе доступность, скорость и разнообразие стилистик, адаптированных под русскоязычную аудиторию. Visionary предлагает минималистичный интерфейс, низкую стоимость подписки и эксклюзивные стили («киберпанк», «аниме», «нуар», «ван-Гог» и другие), заполняя нишу между универсальными редакторами и узкоспециализированными AI-арт-сервисами. В таблице 1 представлено сравнение аналогов по критериям.

Таблица 1 – Обзор аналогов

Параметр	Visionary	PicsArt	DeepArt
Стоимость (Pro)	149Р/месяц	От \$13/месяц	От \$30 за премиум-фильтры
Уникальные стили	Киберпанк, аниме, Ван Гог, нуар, готика и другие	Стандартные и AI-фильтры	Художественные стили (Ван Гог, Пикассо)
Интерфейс	Минималистичный, интуитивный	Перегруженный	Простой, но ограниченный

3 Моделирование системы

3.1 Диаграмма прецедентов (Use Case)

Диаграмма прецедентов – диаграмма, описывающая, какой функционал разрабатываемой программной системы доступен каждой группе пользователей.

На рисунке 1 представлена диаграмма прецедентов для разрабатываемого приложения Visionary. На ней представлены 5 основных роли – неавторизованный пользователь, авторизованный пользователь, пользователь без подписки, пользователь с подпиской Премиум и пользователь с подпиской Про.

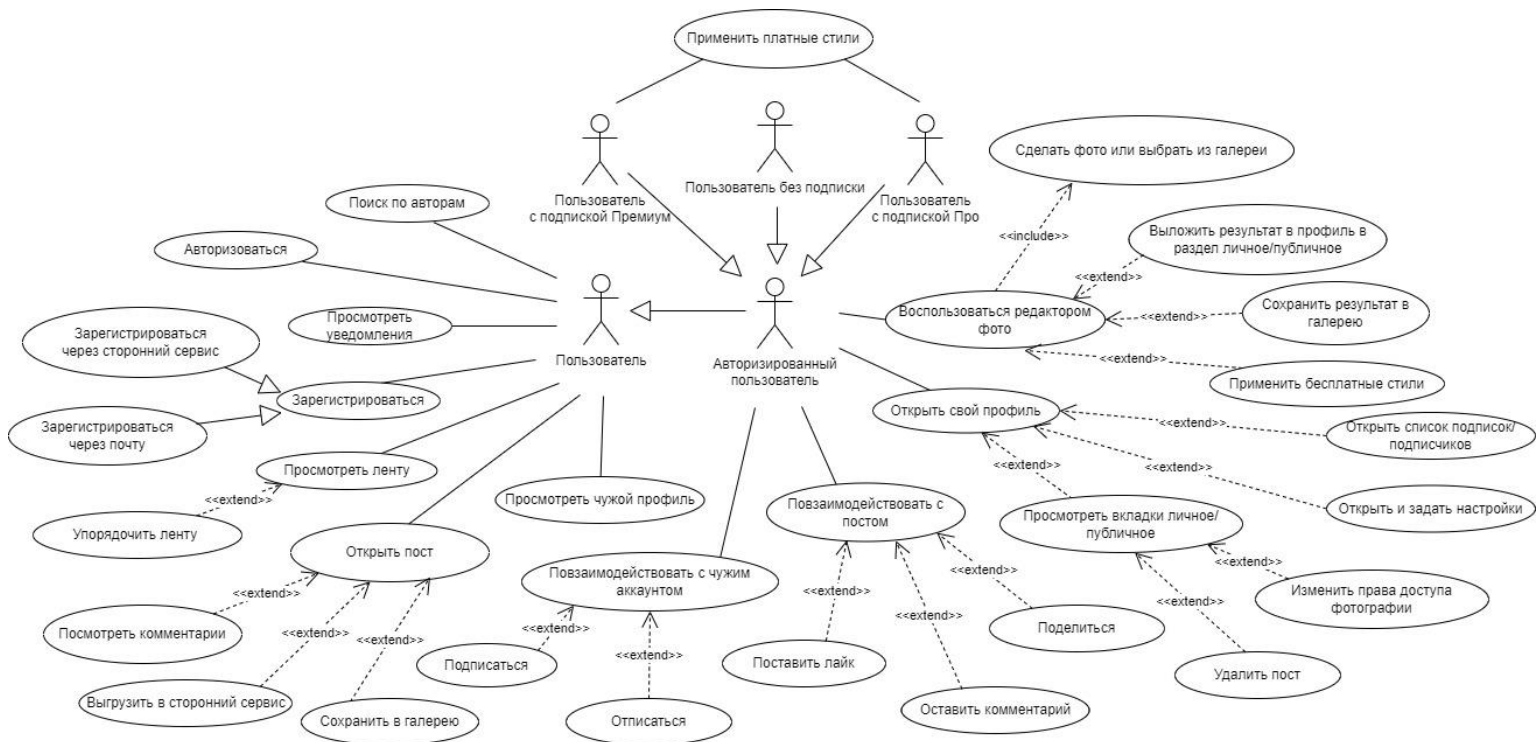


Рисунок 1 – Диаграмма Use-Case

3.2 Диаграмма состояний (Statechart diagram)

Диаграмма состояний определяет последовательность состояний объекта, вызванных последовательностью событий. Данная диаграмма полезна при моделировании жизненного цикла объекта.

На рисунке 2 представлена диаграмма состояний для разрабатываемого приложения Visionary для создания поста внутри приложения.

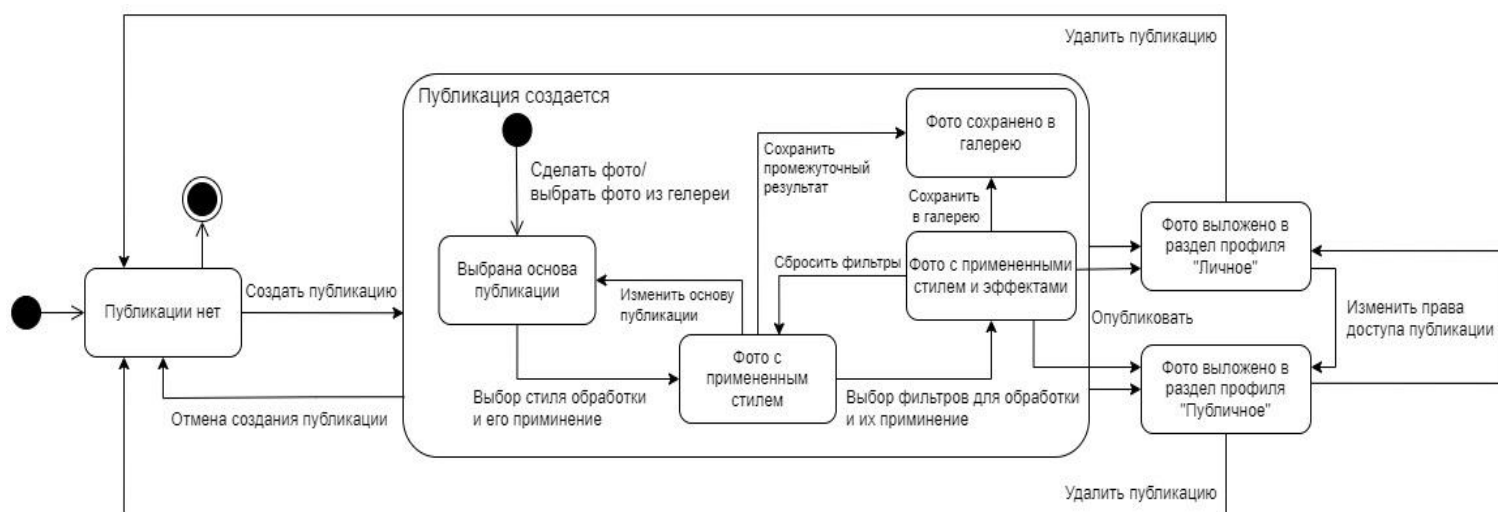


Рисунок 2 – Диаграмма состояний создания публикации

3.3 Диаграмма активностей

Диаграмма активностей – UML-диаграмма, на которой показаны действия, состояния которых описано на диаграмме состояний.

На рисунке 3 представлена диаграмма активностей для разрабатываемого приложения Visionary:

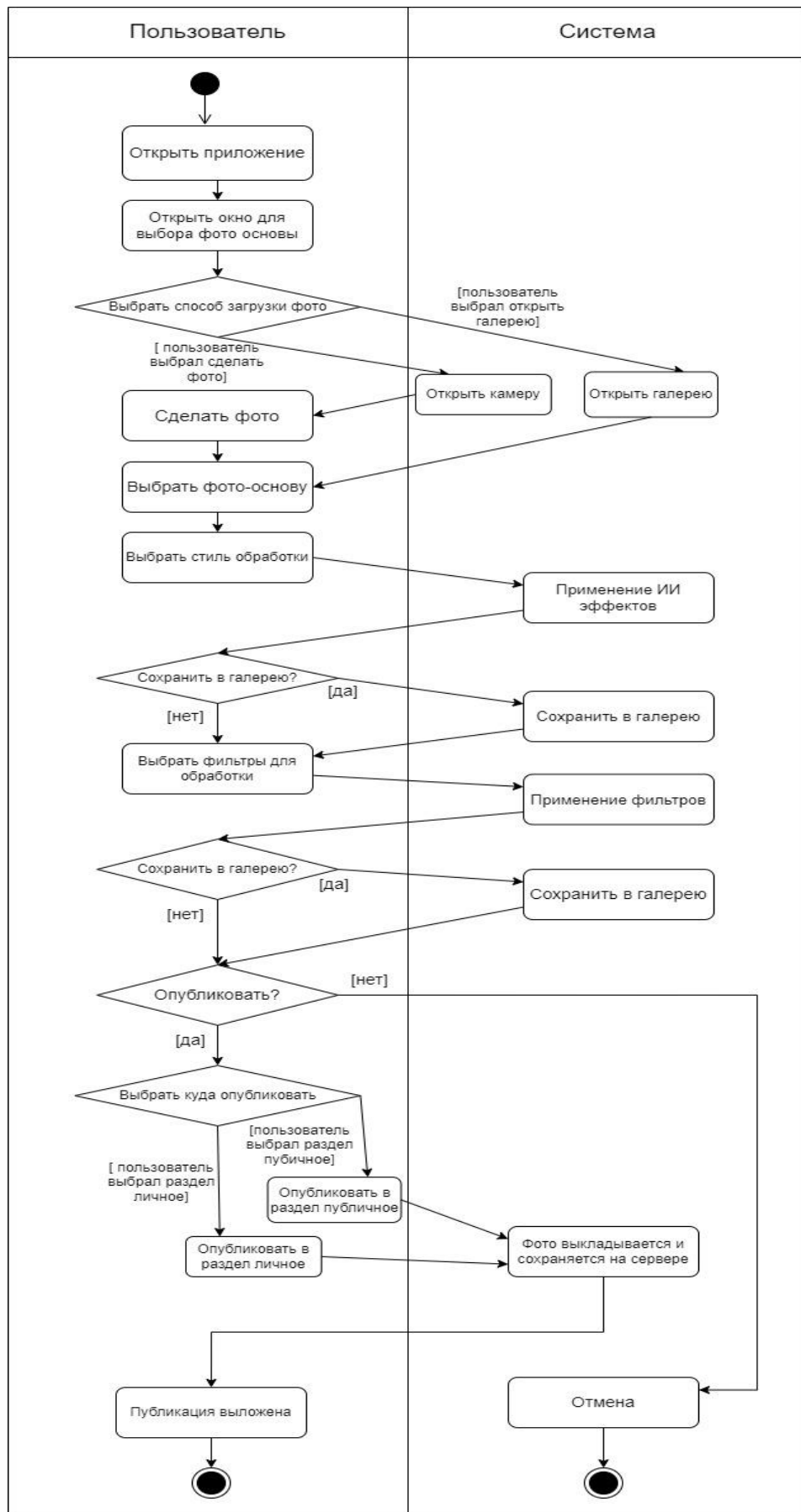


Рисунок 3 – Диаграмма активностей

3.4 ER-диаграмма

ER-диаграмма – это модель данных, визуализирующая связь между «сущностями» внутри системы.

На рисунке 4 представлена ER-диаграмма для разрабатываемого приложения Visionary.

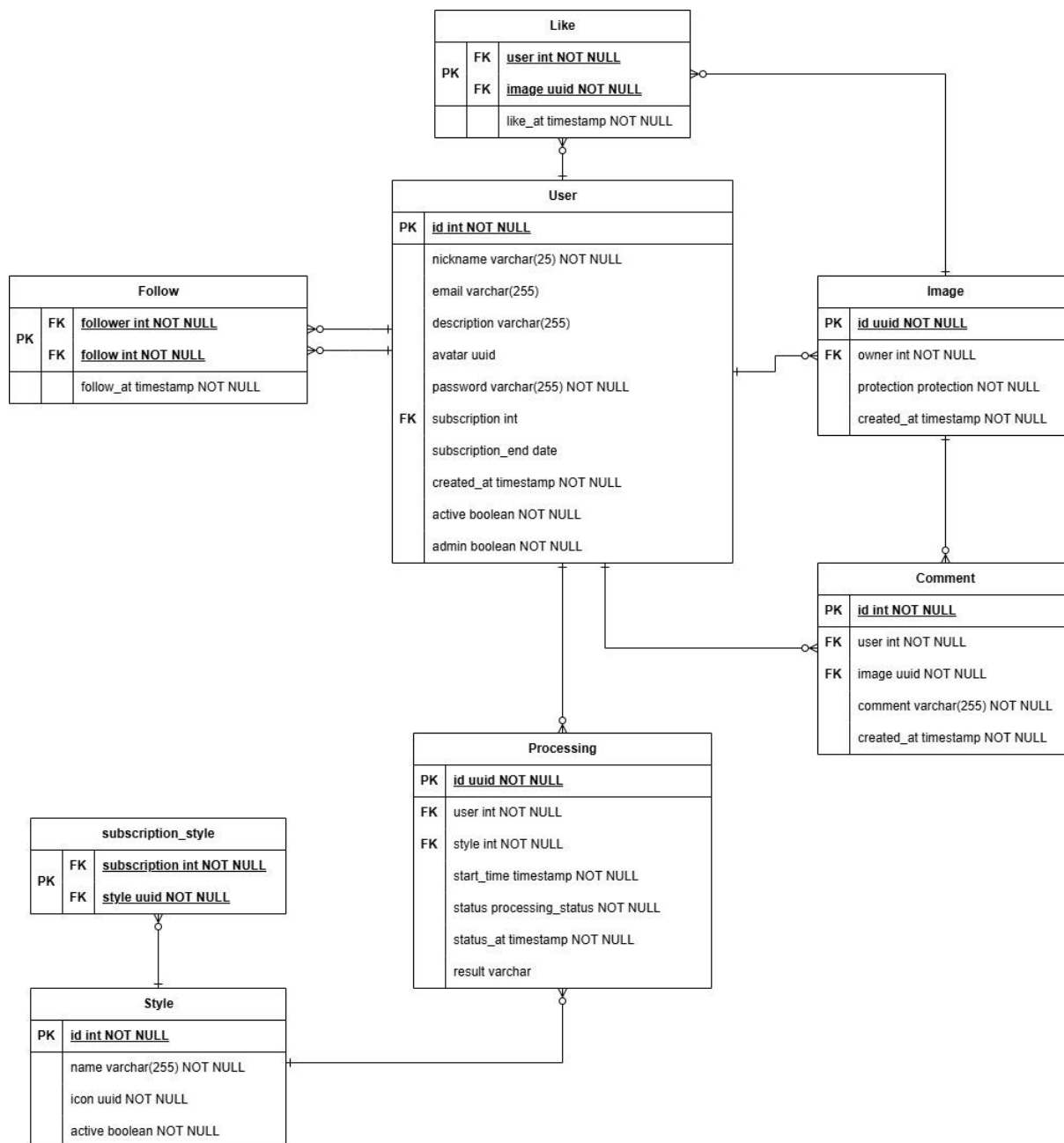


Рисунок 4 – ER-диаграмма

3.5 Функциональная схема

Функциональная схема – это схема, демонстрирующая основные функциональные блоки приложения Visionary, их взаимосвязи и

На рисунке 5 и 6 представлены функциональные схема для разрабатываемого приложения Visionary для неавторизованного и авторизованного пользователя соответственно, для более детального рассмотрения можно перейти на следующую ссылку, ведущую на страницу в Figma

19

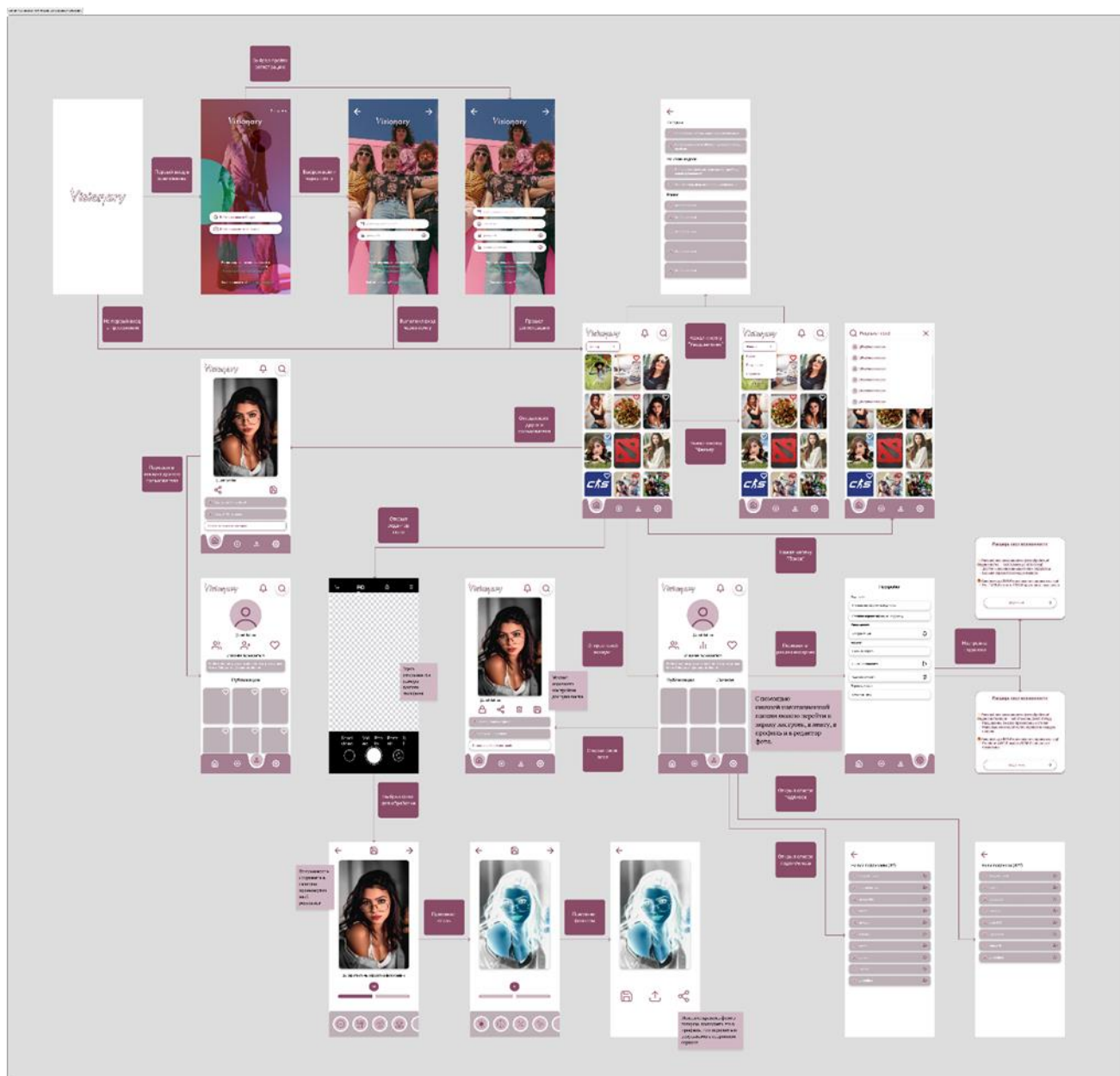


Рисунок 6 – Функциональная схема для авторизованного пользователя

3.6 Диаграмма развертывания (Deployment diagram)

Диаграммы развертывания обычно используются для визуализации физического аппаратного и программного обеспечения системы.

На рисунке 7 представлена диаграмма развертывания для разрабатываемого приложения Visionary.

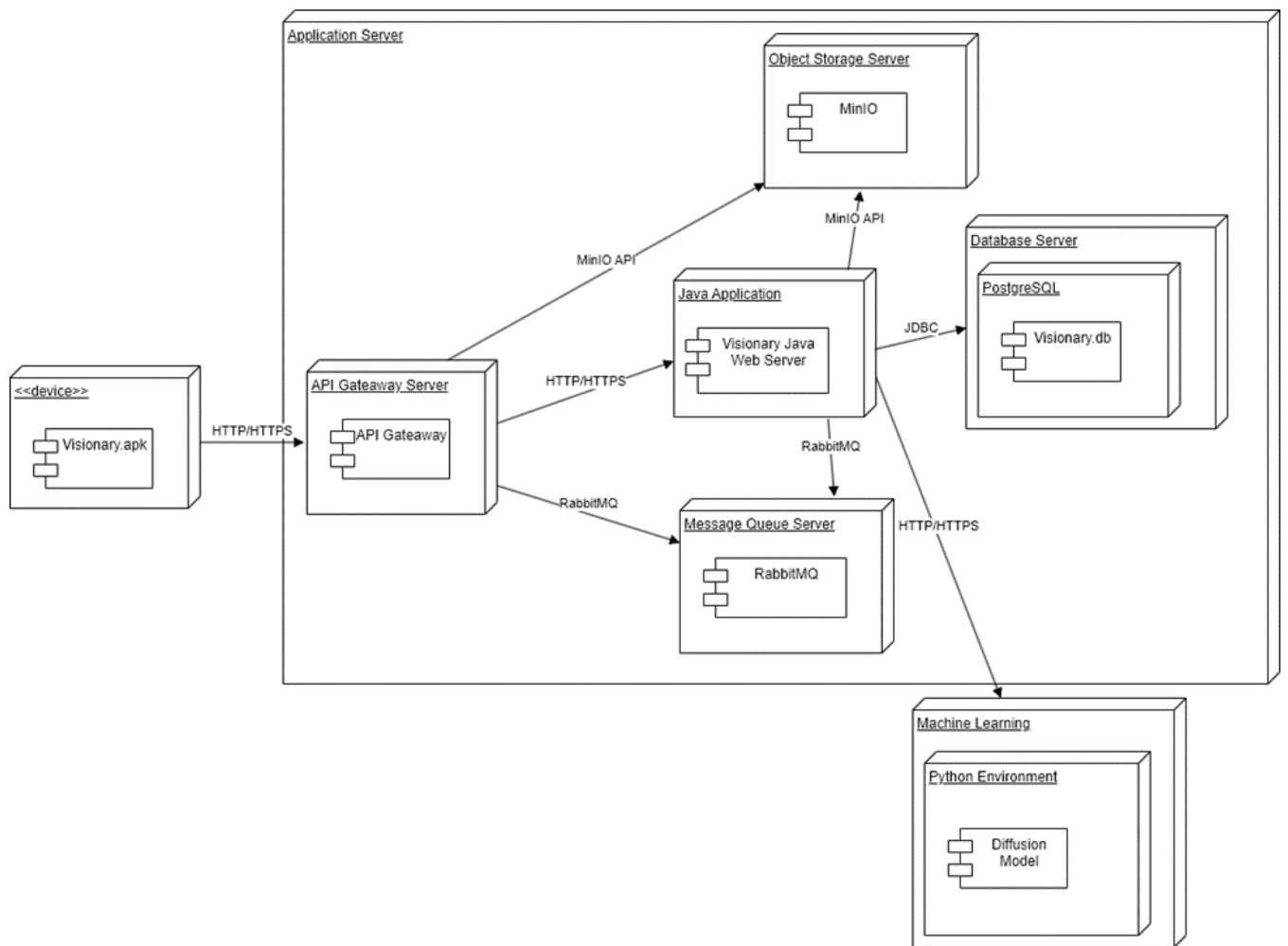


Рисунок 7 – Диаграмма развертывания

4 Реализация

4.1 Средства реализации

Для реализации backend части приложения использовались следующие средства:

- язык программирования Java 19 версии;
- фреймворк для разработки приложений на Java - Spring Framework;
- среда разработки приложений для Java - IntelliJ IDEA.

Для реализации хранения данных приложения использовались следующие средства:

- СУБД PostgreSQL;
- объектное хранилище данных MinIO.

Для реализации клиентской части приложения использовались следующие средства:

- язык программирования Kotlin версия 1.9.23;
- Android Studio Jellyfish | 2023.3.1.

Для реализации клиентской части админ панели использовались следующие средства:

- язык программирования Python 3.12;
- фреймворк Flask версии 3.1

Для реализации серверной части с нейросетевой обработкой:

- Язык программирования Python 3.12;
- Фреймворк FastApi обращения к этой части через api;
- Библиотека Diffusers использования предобученных нейросетевых архитектур.

Для развертывания приложения использовались следующие средства:

- Docker для автоматизации развертывания приложения;
- Процесс CI/CD в GitHub Actions.

В качестве преимуществ выбранных технологий можно отметить следующее:

Для Java и Spring:

- готовые решения для реализации RESTful архитектуры;
- инструменты для работы с PostgreSQL.

Для PostgreSQL:

- предоставляет большой бесплатный функционал;
- надежная и высокопроизводительная.

Для Kotlin:

- мультиплатформенность;
- понятная и полная документация;
- возможность оптимально проектировать мобильные приложения.

Для реализации backend-части приложения выбран язык Java 19 в сочетании с фреймворком Spring и средой разработки IntelliJ IDEA. Это сочетание обеспечивает высокую производительность, надежность и безопасность серверной части, а Spring Framework значительно ускоряет разработку благодаря готовым решениям для REST API и интеграции с базами данных. IntelliJ IDEA как профессиональная среда разработки повышает продуктивность за счет интеллектуального автодополнения кода и встроенных инструментов анализа.

Для хранения данных используется комбинация PostgreSQL и MinIO. PostgreSQL гарантирует целостность данных и поддерживает сложные запросы, что особенно важно для работы с пользовательской аналитикой, а MinIO обеспечивает надежное и масштабируемое хранение медиафайлов с полной совместимостью с S3 API.

Клиентская часть разрабатывалась на Kotlin с использованием Android Studio. Kotlin предлагает современный синтаксис и полную совместимость с Java, что ускоряет разработку мобильного клиента, а Android Studio

предоставляет полный набор инструментов для создания качественных приложений.

Для нейросетевой обработки изображений использовался Python 3.12 с фреймворком FastAPI и библиотекой Diffusers. FastAPI обеспечивает высокопроизводительное API с автоматической генерацией документации, а Diffusers предоставляет доступ к передовым моделям генерации изображений с минимальными затратами на реализацию.

Для админ-панели использовался Python 3.12 с фреймворком Flask, что обеспечивало наибольшее удобство написания и обеспечивало современные инструменты для работы.

4.2 Архитектура клиентской части

Приложение построено на трехуровневой архитектуре: клиент (веб-приложение с админ-панелью и android-приложение) - сервер - база данных

Приложение реализовано в соответствии с подходом MVVM (Model – View – ViewModel). MVVM — это паттерн разработки, позволяющий разделить приложение на три функциональные части:

- Model – основная логика программы;
- View – вид или представление (пользовательский интерфейс), в структуре кода называется screen;
- ViewModel – модель представления, которая служит прослойкой между View и Model.

Шаблон MVVM помогает четко отделять бизнес-логику приложения и логику презентации от пользовательского интерфейса. Поддержание четкого разделения между логикой приложения и пользовательским интерфейсом помогает решить многочисленные проблемы разработки и упрощает тестирование, обслуживание и развитие приложения. В мобильном приложении шаблон MVVM реализуется следующей структурой папок:

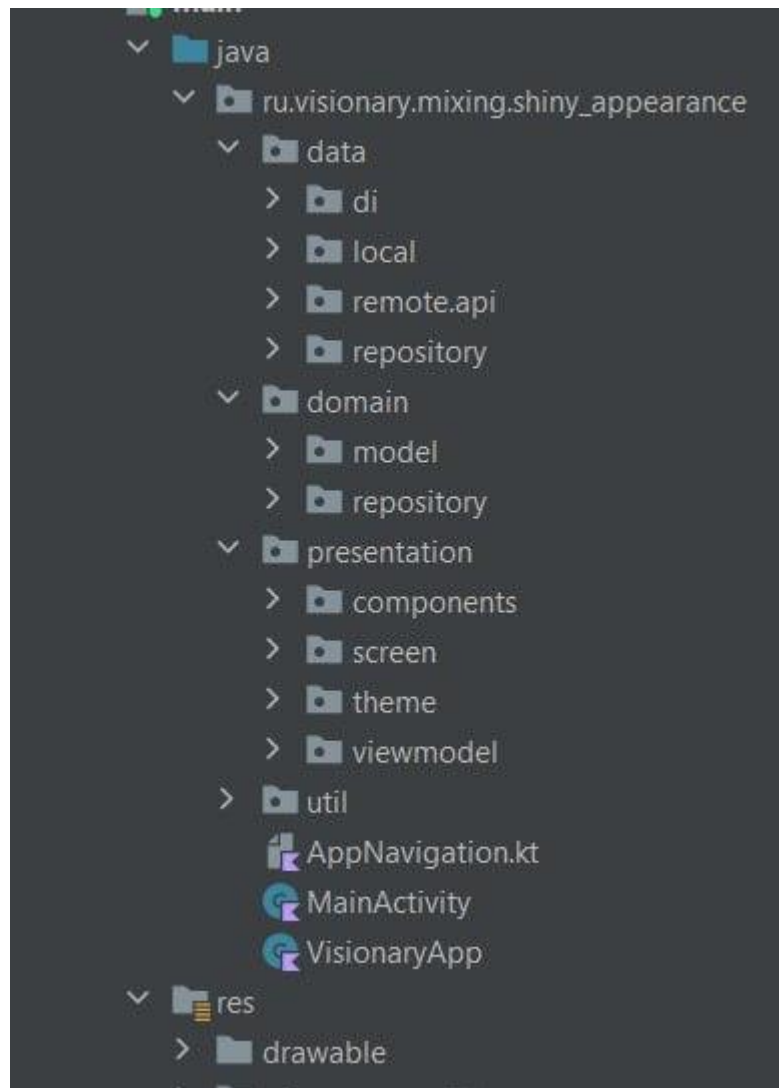


Рисунок 8 – Реализация шаблона MVVM

4.2.1 Компонент представления (View)

Слой screen содержит классы отображения визуальных элементов. Каждый экран, реализуемый в классе с постфиксом «Screen», вынесен в отдельную папку в которой находятся логика для всех экранов. В папке theme находятся все основные шрифты, темы, цвета для использования внутри приложения. Для примера на рисунке показана полная структура папки screen.

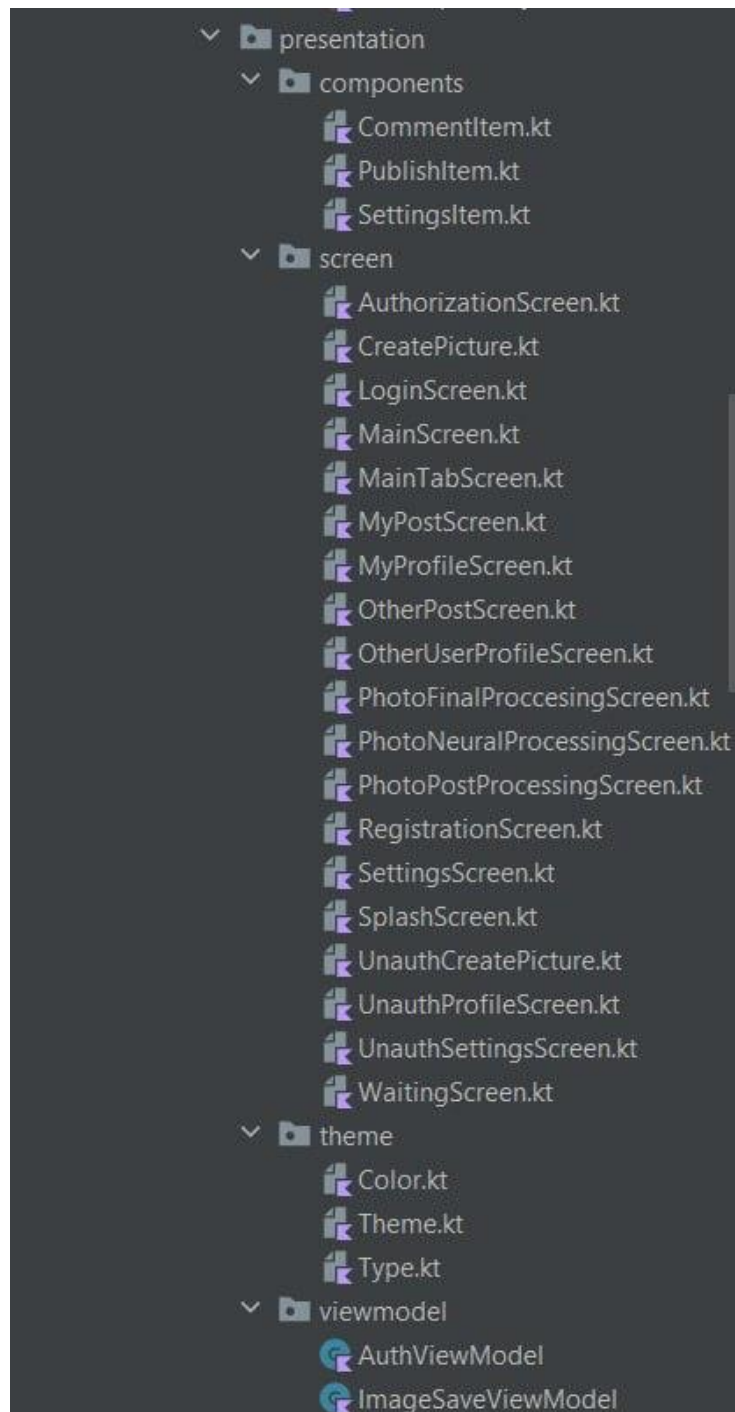


Рисунок 9 – Реализация компонента View

4.2.2 Компонент данных (Model)

Взаимодействие с данными в приложении реализуется в слоях data и model.

Слой data состоит из четырёх частей:

— папка di содержит логику для внедрения зависимостей и представление объектов классов.

- папка `remote.api` содержит классы сервисов для взаимодействия с серверной частью приложения.
- папка `repository` содержит классы реализации репозитория, которые отвечают за получение данных из сервисов, выполняют необходимые преобразования с данными и обрабатывают ошибки, которые могут возникнуть при осуществлении запросов;
- папка `local` содержит классы, которые локально сохраняют данные о пользователях – `access` и `refresh` токены;
- Папка `model` хранит все модели, необходимые для функционирования приложения.

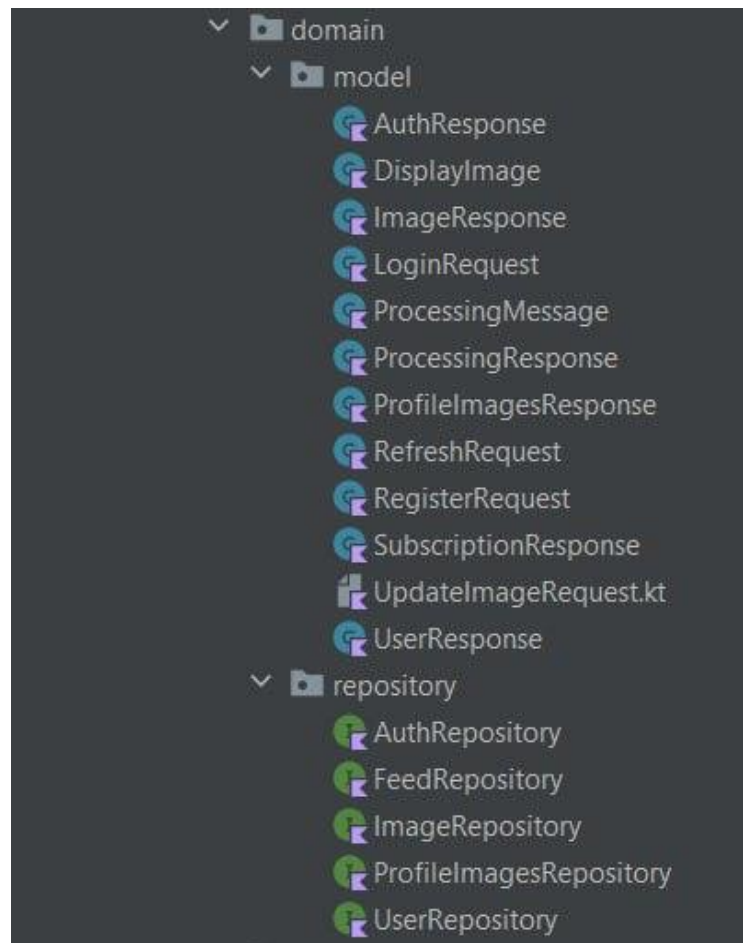


Рисунок 10 – Реализация папки `model`

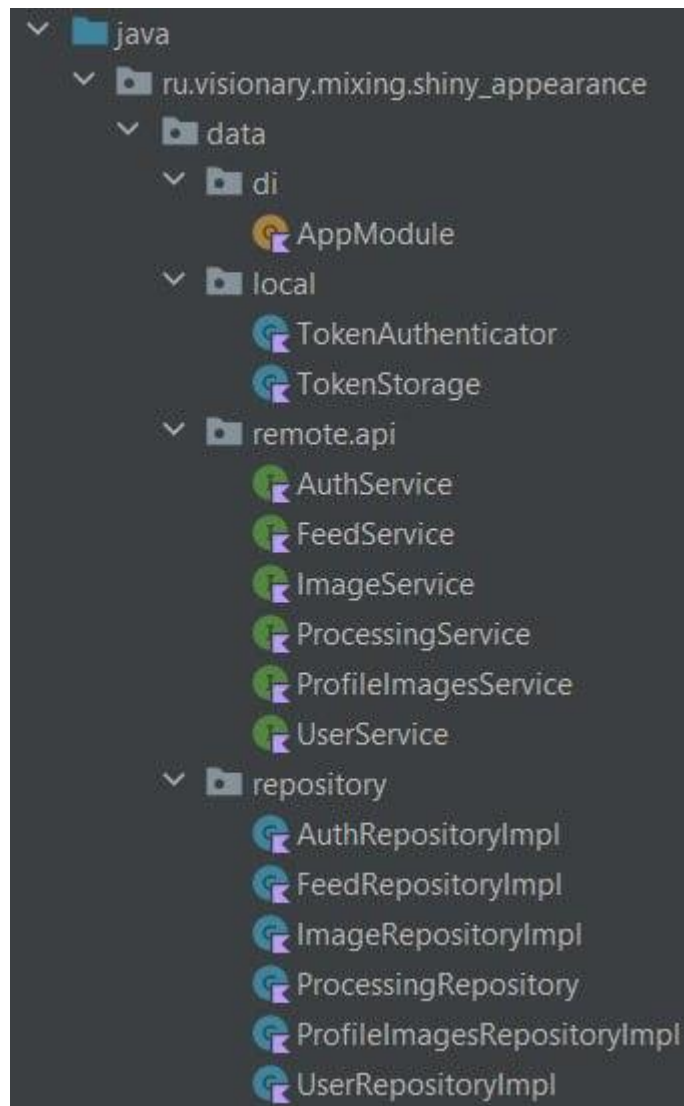


Рисунок 11 – Реализация слоя data

4.2.3 Компонент модели представления (ViewModel)

Слой viewmodels содержит классы, которые реализуют взаимодействие визуального представления из папки screen с методами получения и обработки данных из слоя repository.

В случае, если на странице необходимо осуществить ввод данных для отправки на сервер, в классах viewmodel реализуются методы, которые получают данные со страницы, формируют из этих данных модели и передают в слой репозитория.

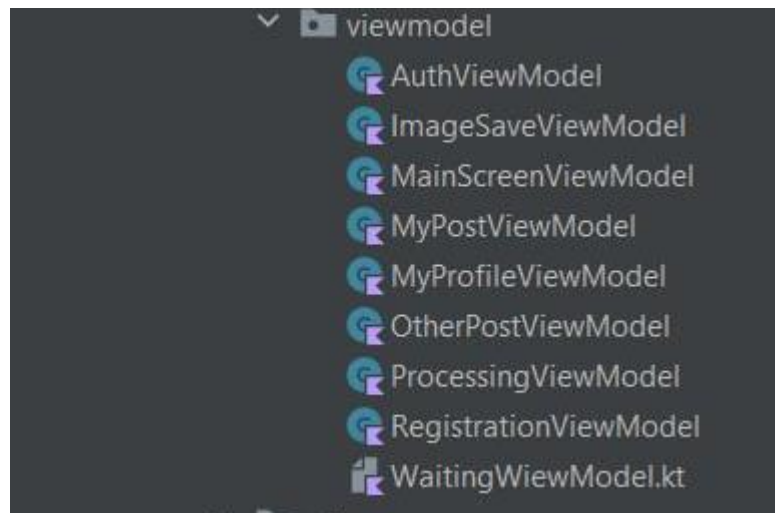


Рисунок 12 – Реализация компонента ViewModel

4.2.4 Слой utils

Данный слой содержит вспомогательные элементы, использующиеся во всём приложении (такие как сохранить изображение в галерею и другие, преобразование изображений и прочее):

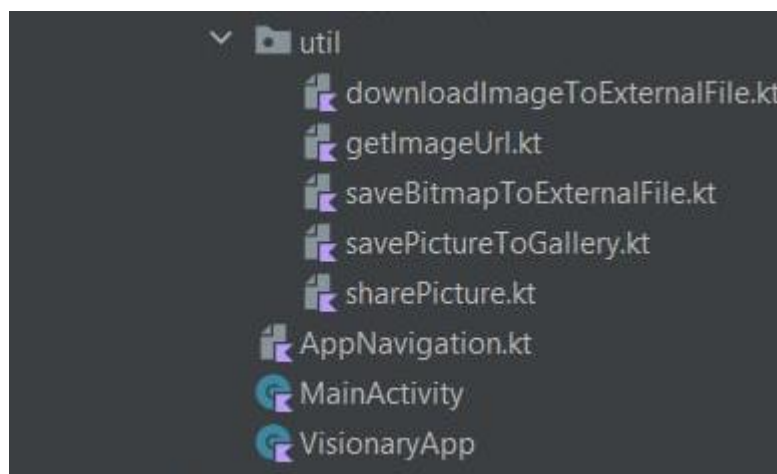


Рисунок 13 – Реализация слоя utils

4.3 Архитектура серверной части

Проект использует архитектурный шаблон Model-View-Controller (MVC), который разделяет логику приложения на три основные компоненты:

— Model — представляет собой данные и бизнес-логику приложения. В этом проекте модель представлена сущностями, такими как классы в пакете entity, которые отражают структуру данных и их поведение;

— View — отвечает за отображение данных пользователю. В данном проекте, скорее всего, за отображение данных и взаимодействие с пользователем отвечает часть приложения, связанная с WebSocket и клиентским интерфейсом (пакет websocket);

— Controller — управляет запросами пользователя и взаимодействует с моделью, обрабатывая запросы и изменяя состояние модели. Контроллеры в проекте находятся в пакете controller и управляют процессом взаимодействия с пользователем, передавая данные модели для отображения во вью.

4.3.1 Структура проекта

Структура проекта выглядит следующим образом:

— client — содержит логику, связанную с клиентской частью приложения;

— config — конфигурационные файлы, включая настройки для работы приложения, такие как настройки безопасности и базы данных;

— controller — пакет, в котором находятся классы, реализующие контроллеры, отвечающие за обработку запросов и взаимодействие с моделью;

— entity — содержит классы, представляющие сущности и объекты, с которыми работает приложение. Это основа для хранения данных;

— exception — обработка исключений, которые могут возникать в процессе работы приложения;

— job — возможно, связанный с выполнением фоновых задач или интеграцией с планировщиками заданий;

- repository — интерфейсы и классы, связанные с доступом к данным, включая операции с базой данных;
- security — настройки безопасности и защиты данных;
- service — бизнес-логика, которая обрабатывает данные, полученные от контроллеров, и взаимодействует с моделями;
- utils — вспомогательные утилитарные классы, предоставляющие вспомогательные функции для других частей приложения;
- websocket — реализация WebSocket для обмена данными в реальном времени между клиентом и сервером.

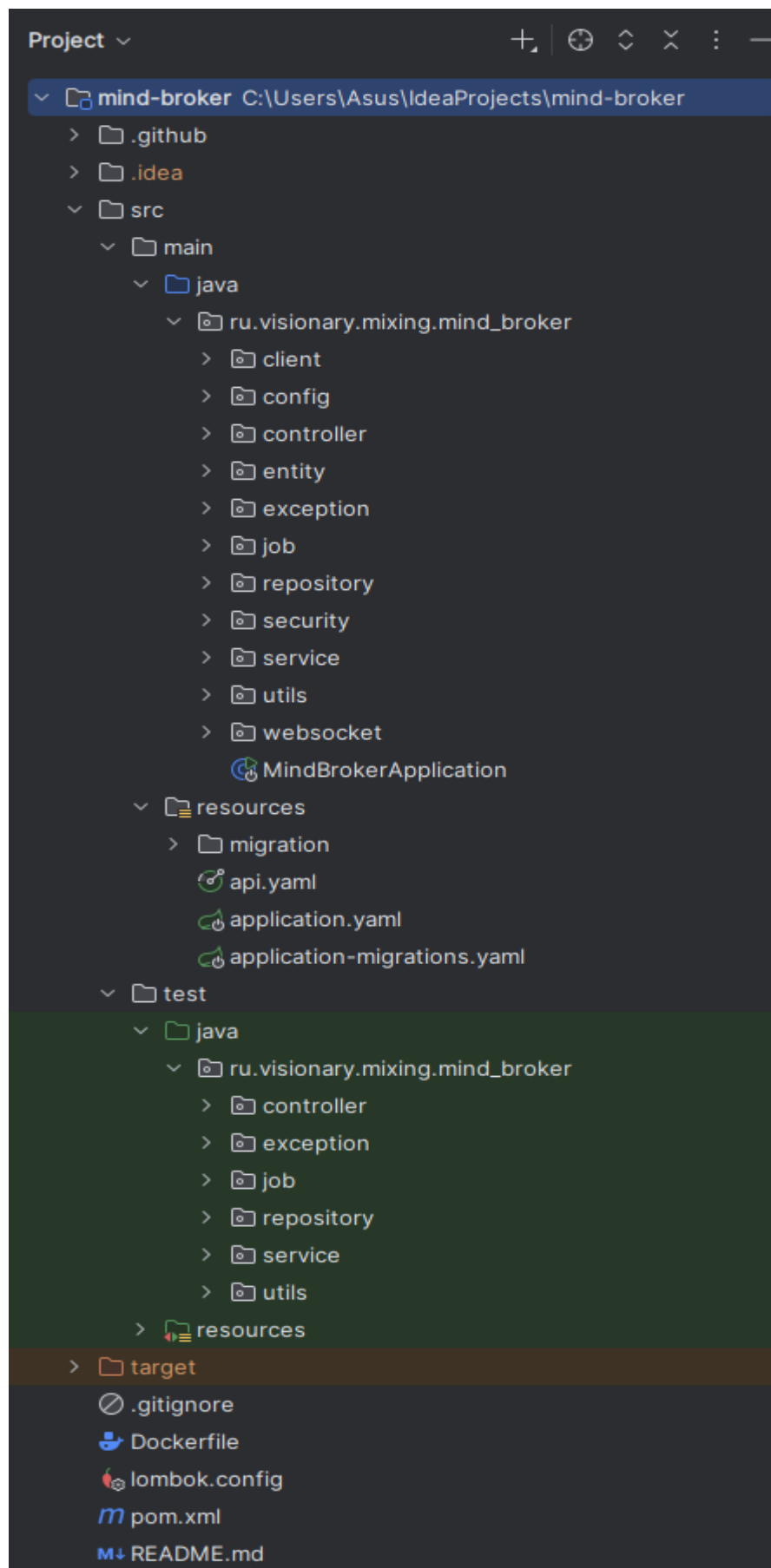


Рисунок 14 – Структура проекта

4.4 Реализация функциональных требований приложения

4.4.1 Функции для пользователей

В ходе разработки приложения были реализованы следующие функции для пользователей:

- авторизоваться в приложение;
- зарегистрироваться в приложение через почту;
- просматривать посты в ленте приложения;
- упорядочить ленту постов;
- открывать посты в ленте приложения;
- читать комментарии к постам;
- сохранять фотографии из постов в галерею телефона;
- экспортировать фото из постов в другие приложения;
- просматривать профиль другого пользователя;
- сохранять фотографии из постов в галерею телефона;
- экспортировать фото из постов в другие приложения.

Согласно техническому заданию не было реализовано:

- зарегистрироваться в приложение через сторонний сервис
- выполнить поиск авторов по имени;

Итого, из функций для пользователей было реализовано 11 функций из 13, основной функционал для работы приложения был реализован и приложение работает без критических ошибок.

4.4.2 Функции для авторизованных пользователей

Авторизованный пользователь обладает теми же функциями, что и обычный пользователь, а также следующими нижеперечисленными.

В ходе разработки приложения были реализованы следующие функции для авторизованных пользователей:

Работа с профилем:

- открывать свой профиль;

- просматривать вкладки личное и публичное в профиле;
- редактировать профиль (изменение имени, пароля);
- изменять уровень доступа (личный или публичный) для своих фотографий;
- подписываться на других авторизованных пользователей;
- отписываться от других авторизованных пользователей;
- просматривать список своих подписок/подписчиков;
- выйти из аккаунта;
- удалить профиль.

Обработка и работа с фотографиями:

- использовать редактор фото;
- применять фильтры к фотографиям перед или после стилизации (яркость, контрастность);
- делать фотографии и выбирать их из галереи телефона для загрузки в редактор;
- обрабатывать фото с применением бесплатных стилей;
- сохранять результат обработки в галерею телефона;
- экспортировать результат обработки в другие приложения;
- выкладывать результат обработки фото в профиль в разделы личное или публичное.

Взаимодействие с контентом:

- ставить лайк на понравившийся пост;
- оставлять комментарии под постами.

Согласно техническому заданию не было реализовано:

- удалять посты из вкладок личное и публичное в профиле;
- работа с уведомлениями;
- возможность приобретение платной подписки.

Итого, из функций для авторизованных пользователей было реализовано 18 функций из 21. В ходе работы были реализованы основные функции для

работоспособности приложения, не реализованные же функции не несли важную составляющую в общую работоспособность. Также в ходе работы над приложением, было решено отказаться от интеграции платных подписок для повышения аудитории приложения и, следовательно, увеличения обратной связи от пользователей и большей клиент ориентированности.

4.4.3 Функции для администратора

В ходе разработки приложения были реализованы следующие функции для администратора:

Управление контентом:

- удаление фотографий или комментариев;
- удаление аккаунта пользователя;
- удаление постов пользователей.

Мониторинг:

Будет производиться согласно с 4 золотыми сигналами мониторинга:

- процент ошибок;
- трафик в секунду/минуту;
- время обработки запроса;
- утилизация.

Помимо планируемого функционала в техническом задании, также было реализовано:

- доступ к базе данных PostgreSQL;
- просмотр документации по API приложения;
- доступ к базе MinIO;
- просмотр метрик в Grafana;
- просмотр метрик и предупреждений в Prometheus;
- просмотр и управление очередями сообщений в RabbitMQ.

Для данной роли были реализованы 4 из 4 функций, в админ-панели реализованы все данные функции и мониторинг реализован с помощью сервиса Grafana с возможностью осуществлять мониторинг всех связанных с приложением процессов.

4.4.4 Требования к работе приложения

В ходе разработки приложения были реализованы следующие требования к удобству использования:

Интерфейс приложения:

- навигация по приложению при помощи панели;
- минимальное количество шагов для выполнения основных действий (загрузка, обработка, сохранение фотографий), в приложении каждое из этих действий реализуется за 1,2 и 4 перехода на различные экраны соответственно при заходе на авторизованный аккаунт.

Адаптивность:

- корректное отображение приложения на устройствах с разным разрешением экрана.

Производительность:

- быстрая обработка фотографий с максимальным временем обработки в 5 минут соблюдена, средняя скорость обработки при проверке ручным тестированием составила около 30 секунд;
- минимальное время загрузки фотографий в приложение до 15 секунд до отображения в приложении соблюдено;
- быстрые запросы к серверу и от него для получения информации – из-за трудностей оценки необходимой скорости для запросов к серверу, проверка была проведена только на общую корректность исполняемого кода, и проверка не выявила проблем оптимизации.

В ходе разработки приложения были реализованы следующие требования к безопасности:

Защита данных пользователей:

- использование JWT-токенов для авторизации;
- защита от SQL-инъекций и других видов атак;
- шифрование персональных данных пользователей.

Согласно техническому заданию в требованиях по работе приложения не было реализовано:

- возможность настройки доступа к постам пользователей (открытый/закрытый);

Итого было реализовано 10 из 11 задач. В результате основные требования для работы приложения были реализованы и приложение работает без критических ошибок и с хорошей скоростью выполнения своих функций.

4.5 Нефункциональные требования

Нефункциональные требования определяют качественные характеристики системы, которые не связаны напрямую с функциональностью, но влияют на общее качество и удобство использования приложения. Ниже приведены реализованные нефункциональные требования для разрабатываемого приложения.

4.5.1 Безопасность

Аутентификация и авторизация:

- использование JWT-токенов для авторизации пользователей;
- ограничение времени жизни токенов для предотвращения несанкционированного доступа в 15 минут для access токена и 30 дней для refresh токена;

— защита от атак в виде SQL-инъекции, XSS (межсайтовый скриптинг) и CSRF (межсайтовая подделка запросов).

Шифрование данных:

— шифрование персональных данных пользователей (пароли, электронные почты) с использованием шифрования солью.

Логирование и мониторинг:

- логирование всех неудачных попыток входа в систему;
- логирование попыток доступа к защищенным ресурсам.

4.5.2 Производительность

Время отклика:

— время обработки фотографии нейросетью не превышает 5 минут, а средняя скорость в ходе ручного тестирования оказалась около 30 секунд;

— время загрузки фотографий в приложение (до его отображения на странице) — не более 5 секунд;

— время отклика сервера на запросы — не более 3000 мс — из-за трудностей оценки необходимой скорости для запросов к серверу, проверка была проведена только на общую корректность исполняемого кода, и проверка не выявила проблем оптимизации.

Масштабируемость:

— приложение может поддерживать до 20 одновременных пользователей;

— система обладает возможностью масштабирования для поддержки большего количества пользователей в будущем.

Оптимизация ресурсов:

- минимизировано использование оперативной памяти и процессора на устройствах пользователей – в ходе ручных тестов оно не превысило 35% использования ресурсов телефона;

- имеется оптимизированная загрузка изображений для уменьшения потребления трафика.

4.5.3 Совместимость

Поддержка операционных систем:

- приложение корректно работает на устройствах с операционной системой Android 8.0 и новее.

Корректное отображение в следующих браузерах для админ-панели:

- Microsoft Edge 110.0.1587.57;
- Google Chrome 110.0.5481.177;
- Mozilla Firefox 111.0;
- Yandex Browser 23.1.5.

Интеграция с внешними сервисами:

- возможность экспорта фотографий в сторонние приложения (например, в Instagram, Facebook или WhatsApp).

4.5.4 Гибкость

Поддержка различных устройств:

- приложение корректно работает на устройствах с разным объемом оперативной памяти (от 2 ГБ);

- поддержка устройств с различной производительностью процессора.

Расширяемость:

- возможность добавления новых стилей обработки фотографий без изменения основной архитектуры приложения;
- возможность интеграции новых фильтров и инструментов редактирования.

Энергопотребление:

- приложение должно минимизировать потребление энергии на мобильных устройствах (не превышать 50% от общего потребления батареи при 100% активности).

Согласно техническому заданию в нефункциональных требованиях по работе приложения было реализовано всё из запланированного на основании проведенного ручного тестирования на 5 устройствах.

4.6 Реализация интерфейса приложения

4.6.1 Загрузочный экран

Каждый раз при заходе в приложении первое, что видит пользователь — это загрузочным экран, на котором находится название приложения «Visionary».



Рисунок 15 – Загрузочный экран

4.6.2 Начальный экран

Если пользователь до этого не проходил регистрацию в приложении или вышел из своего аккаунта, то он попадёт на этот экран. На экране сверху расположена кнопка «Пропустить» для перехода на общую ленту приложения и доступу к основным экранам в роли неавторизованного пользователя, в верхней части экрана также расположено название приложения. Внизу же экрана расположены кнопки «Войти с помощью Google», которая

предоставляет возможность войти или создать аккаунт в приложении при помощи данных учетной записи Google, и кнопка «Войти в аккаунт с эл. почтой» для стандартного входа в приложение, с использованием почты и пароля для аутентификации. Также в самом низу расположена кнопка для регистрации аккаунта – «Зарегистрироваться».

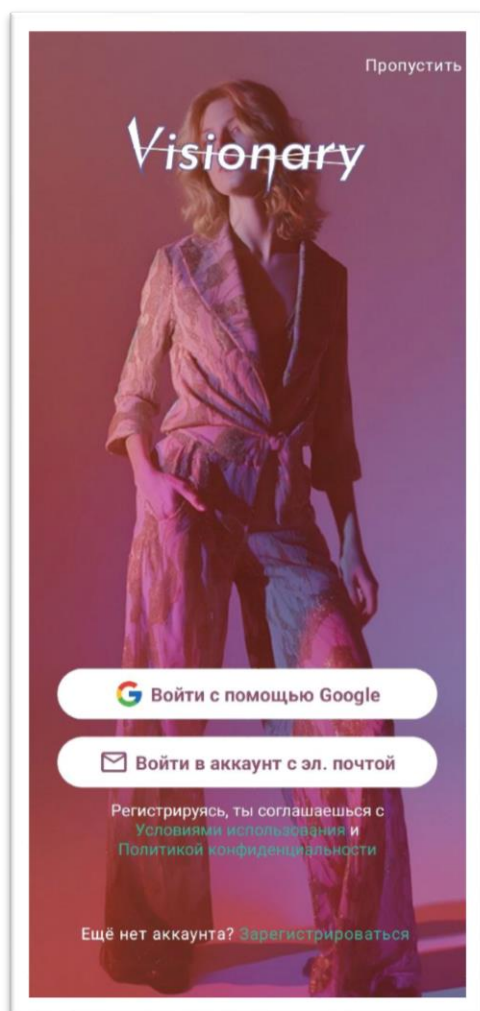


Рисунок 16 – Начальный экран

4.6.3 Экран входа

На странице входа в приложение расположены:

- Стрелки влево и вправо для перехода на предыдущую страницу и авторизации в приложение соответственно, находятся в самом верху экрана по краям;
- название приложения сверху;
- поле для ввода почты в середине экрана;

- поле для ввода пароля ниже середины экрана;
- кнопка «Зарегистрироваться», для перехода на экран для создания нового аккаунта, в самом низу экрана.

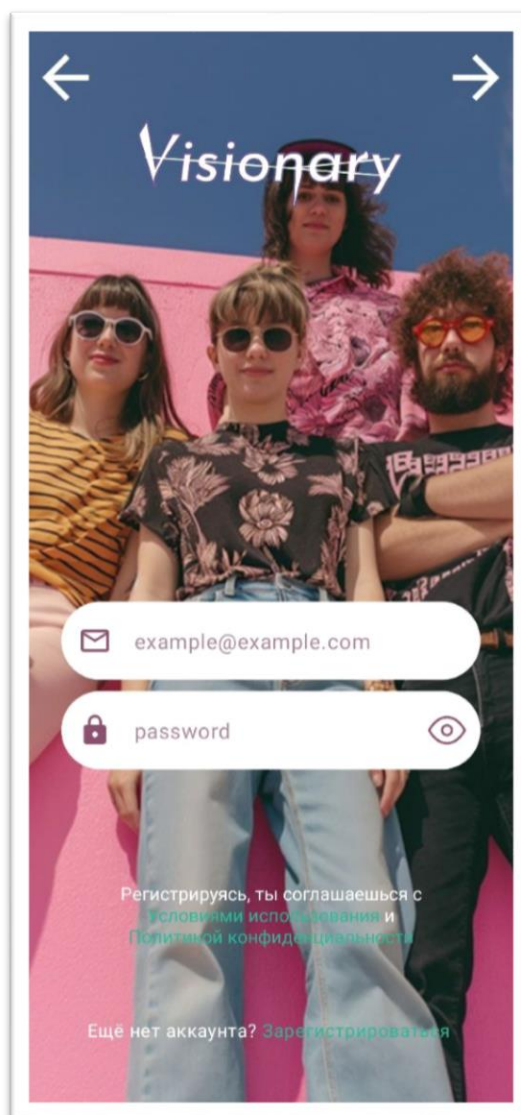


Рисунок 17 – Экран входа в приложение

4.6.4 Экран регистрации

На странице регистрации в приложение расположены:

- Стрелки влево и вправо для перехода на предыдущую страницу и переходом на страницу входа с созданием аккаунта соответственно, находятся в самом верху экрана по краям;
- название приложения сверху;
- поле для ввода почты в середине экрана;

- поле для ввода имени пользователя в середине экрана;
- поле для ввода пароля и его подтверждения ниже середины экрана;
- кнопка «Войти», для перехода на страницу для входа в аккаунт, в самом низу экрана.

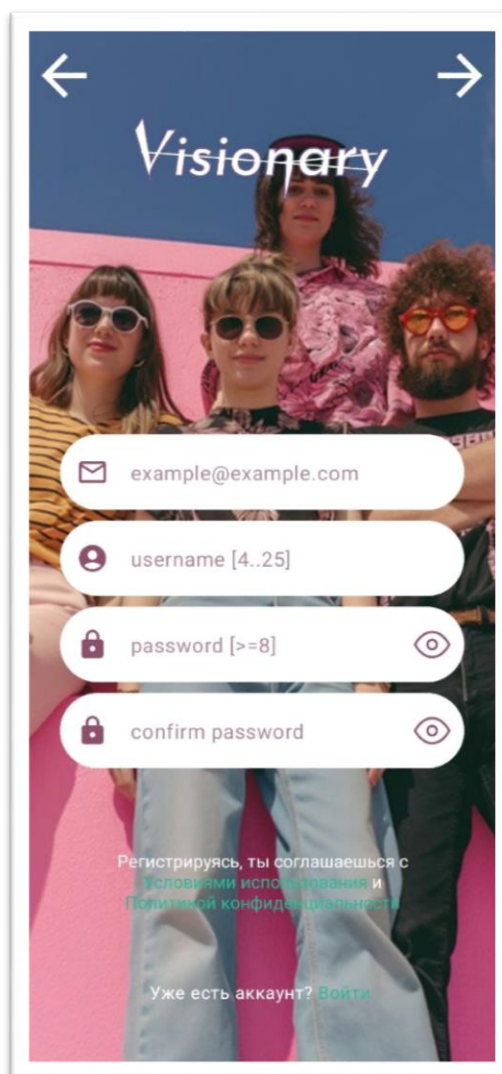


Рисунок 18 – Экран регистрации

4.6.5 Страница ленты с постами

Страница доступна всем пользователям. В верхней шапке страницы расположено название приложения, кнопка просмотра уведомлений и кнопка поиска постов по имени автора. На странице можно увидеть все посты, которые были опубликованы в приложение за всё время. Помимо этого, снизу находится панель навигации с кнопками для перехода на следующие страницы:

- лента с постами – иконка здания;
- выбор изображения для обработки – иконка плюса;
- личный кабинет – иконка человека;
- настройки – иконка шестерёнки.

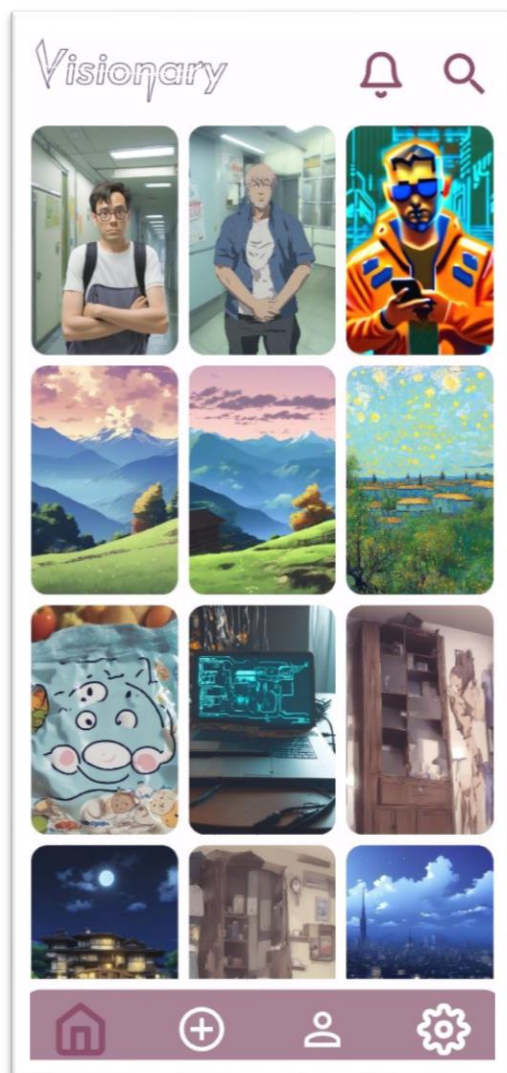


Рисунок 19 – Страница ленты с постами

На каждый пост можно нажать и откроется страница этого поста. На этой странице присутствует такая же верхняя шапка и панель навигации. По центру отображается картинка из поста, на который мы перешли. Слева под картинкой расположен никнейм автора картинки. Ниже расположены кнопки:

- лайкнуть картинку – иконка сердца;

— поделиться картинкой в сторонних приложениях — иконка соединений;

— сохранить картинку на устройстве — иконка дискеты.

Также под постом можно видеть все оставленные комментарии и можно оставить свои комментарий, написав их в поле «Оставьте свой комментарий».

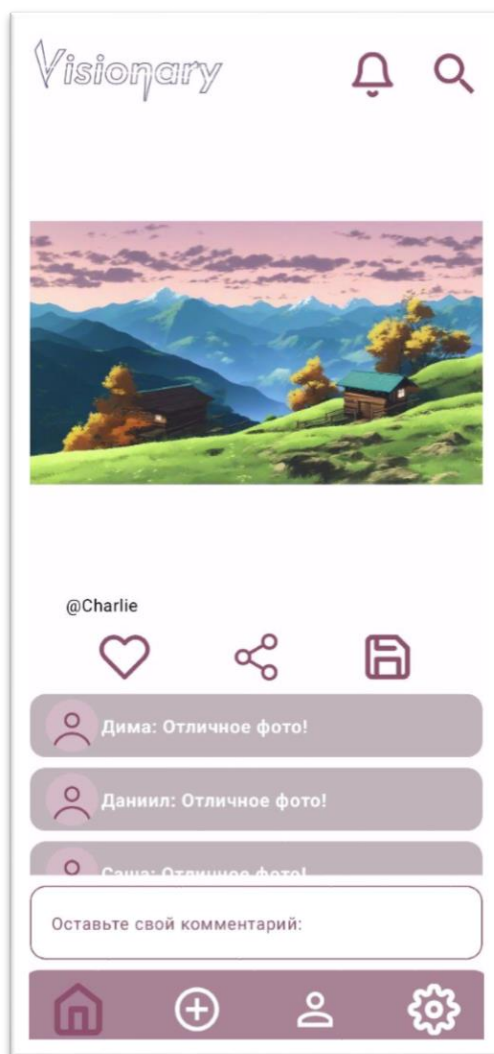


Рисунок 20 – Страница поста в ленте

4.6.6 Страницы для обработки изображений

Страница доступна только авторизованным пользователям. Перейдя на вкладку выбор изображения для обработки из панели навигации, возможен выбор изображения, которое мы хотим отдать в обработку, также возможно сделать фотографию с камеры телефона.



Рисунок 21 – Экран для выбора изображения для обработки

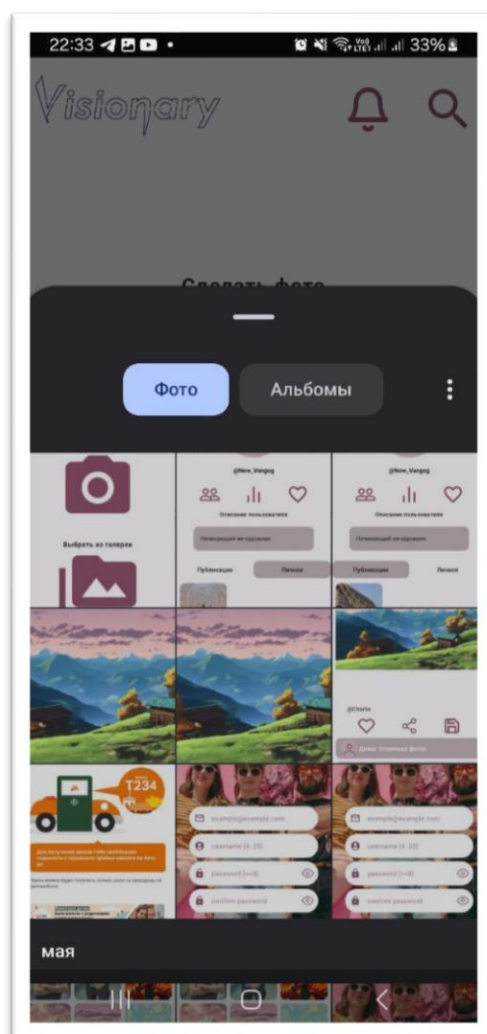


Рисунок 22 – Всплывающее окно с выбором изображения

После того, как было выбрано или сделано изображение открывается окно с ним. В верхней части расположены стрелки направо и налево, которые отвечают за переход на экран с выбором изображения и за начало обработки изображения нейросетью в выбранном стиле соответственно. Снизу изображения расположена кнопка для его сохранения в галерею – иконка дискеты. В нижней части экрана расположены слайдер для регулирования влияния нейросети на изображения и сами нейросетевые стили обработки:

- «аниме»;
- «гибли» - стилистика аниме;
- «киберпанк»;
- «нуар»;

— «ван-Гог».

Все эти стили доступны только при наличии оформленной у пользователя подписки Премиум, при наличии подписки Премиум доступны только «аниме», «киберпанк» и «гибли». При отсутствии у пользователя подписки доступен только стиль «аниме».

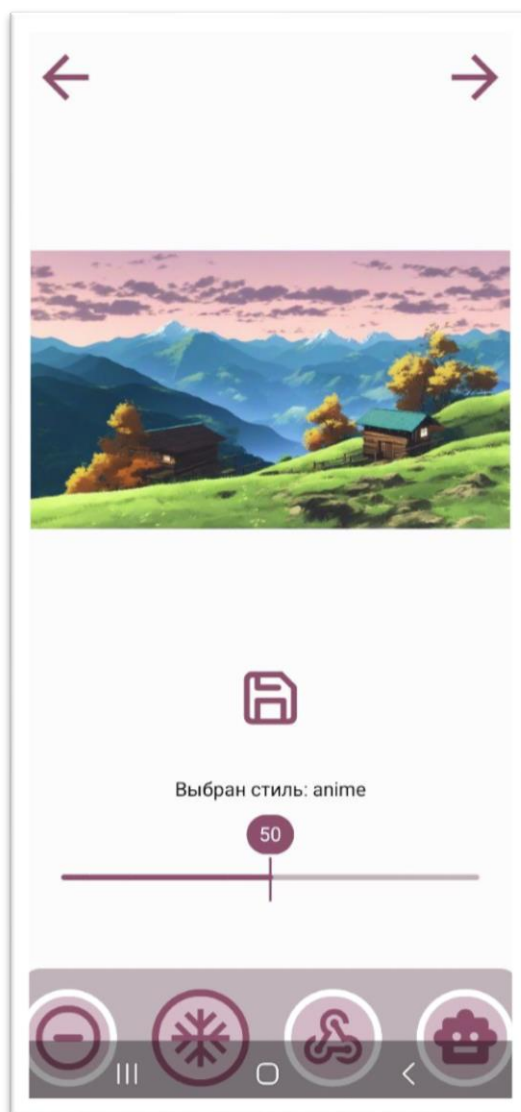


Рисунок 23 – Экран выбора стиля обработки изображения

После выбора стиля обработки и нажатия на стрелку вправо начинается процесс обработки, которые в среднем длится около 30 секунд. Во время обработки, можно отменить процесс, чтобы вернуться на предыдущий экран.



Рисунок 24 – Процесс обработки

Страница с применением фильтров доступна только авторизованным пользователям. После успешной обработки или пропуска обработки нейросетью пользователю доступны следующие функции:

- возвращение на страницу с выбором стиля обработки – иконка стрелка влево;
- переход к странице для выбора опции экспорта изображения – иконка стрелка вправо;
- сохранить изображение с фильтрами в галерею – иконка дискеты;
- слайдер для регулирования силы применения фильтра;
- фильтры обработки фото – яркость, контрастность, обрезка, резкость.

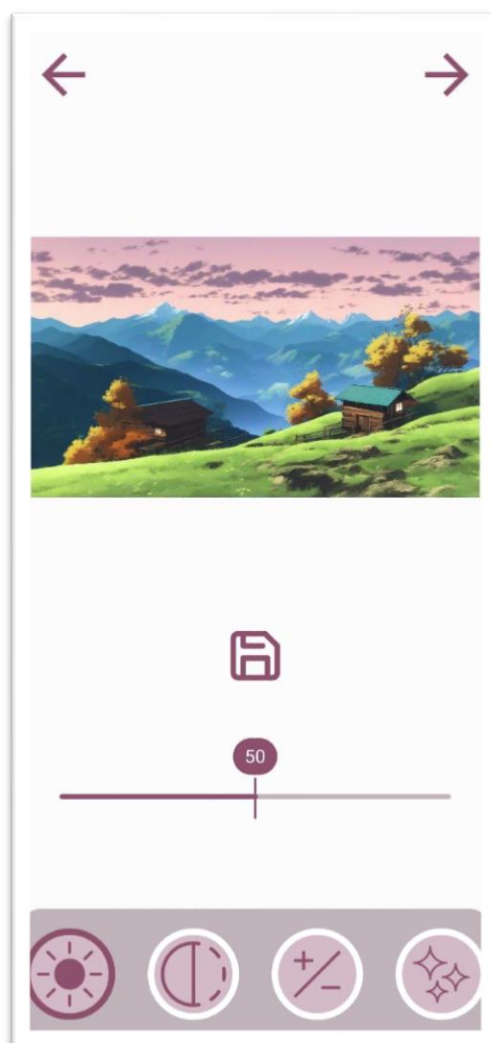


Рисунок 25 – Экран выбора фильтров для изображения

Следующей страницей после выбора фильтров является страница выбора типа экспорта изображения. Имеются три опции:

- сохранение изображения локально на устройстве – иконка дискеты;
- публикация изображения внутри приложения в личное или публичное – иконка со стрелкой вверх;
- поделиться картинкой в сторонних приложениях – иконка соединений;

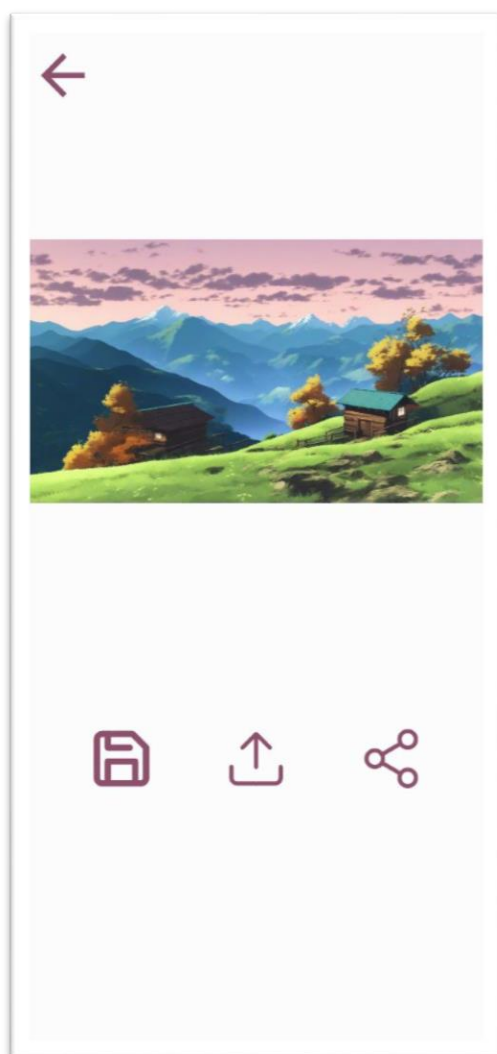


Рисунок 26 – Страницы выбора типа экспорта изображения

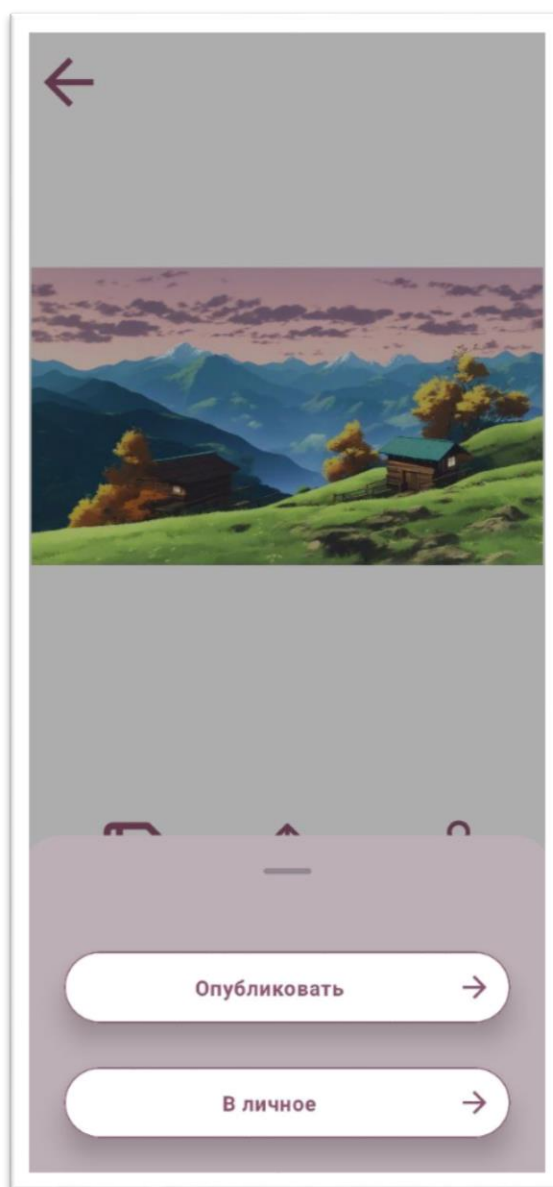


Рисунок 27 – Всплывающее окно при нажатии кнопки публикации

4.6.7 Страница личного кабинета

Страница доступна только авторизованным пользователям. Перейдя на соответствующую вкладку в панели навигации, становится видимым основная информация об аккаунте. На странице расположены:

- аватарка пользователя в верхней части экрана;
- никнейм пользователя, расположенный под аватаркой;
- информация о подписчиках, иконка двух людей;
- информация о подписках на другие аккаунты, иконка вертикальных линий;

- информация о поставленных автору лайках, иконка сердца;
- описание профиля, расположенное по центру экрана, доступное к редактированию;
- две вкладки — личное и публичное, для просмотра всех выложенных в ленту постов и изображений, сохранённых в приложении.

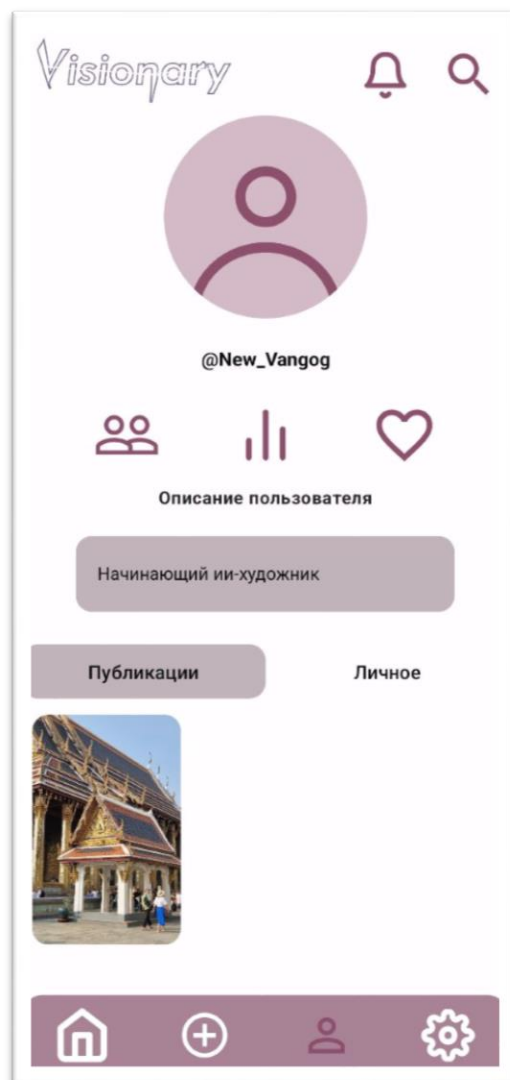


Рисунок 28 – Страница личного кабинета



Рисунок 29 – Страница личного кабинета

4.6.8 Страница настроек

Страница доступна всем пользователям. Перейдя на соответствующую вкладку в панели навигации становится возможным изменение основных настроек. На странице расположены:

- кнопка для просмотра информации и дальнейшего оформления подписки Pro;
- кнопка для просмотра информации и дальнейшего оформления подписки Premium;
- кнопка для включения/выключения уведомлений в приложении;
- кнопка для смены пароля к аккаунту, при нажатии появляется поле для ввода нового пароля и подтверждение;

- кнопка «выйти из аккаунта» для перехода на начальный экран;
- кнопка удалить аккаунт для очищения всех данных о пользователя из приложения и переход на начальный экран
- кнопка для смены темы визуализации приложения, с возможностью выбора темных и светлых цветов.

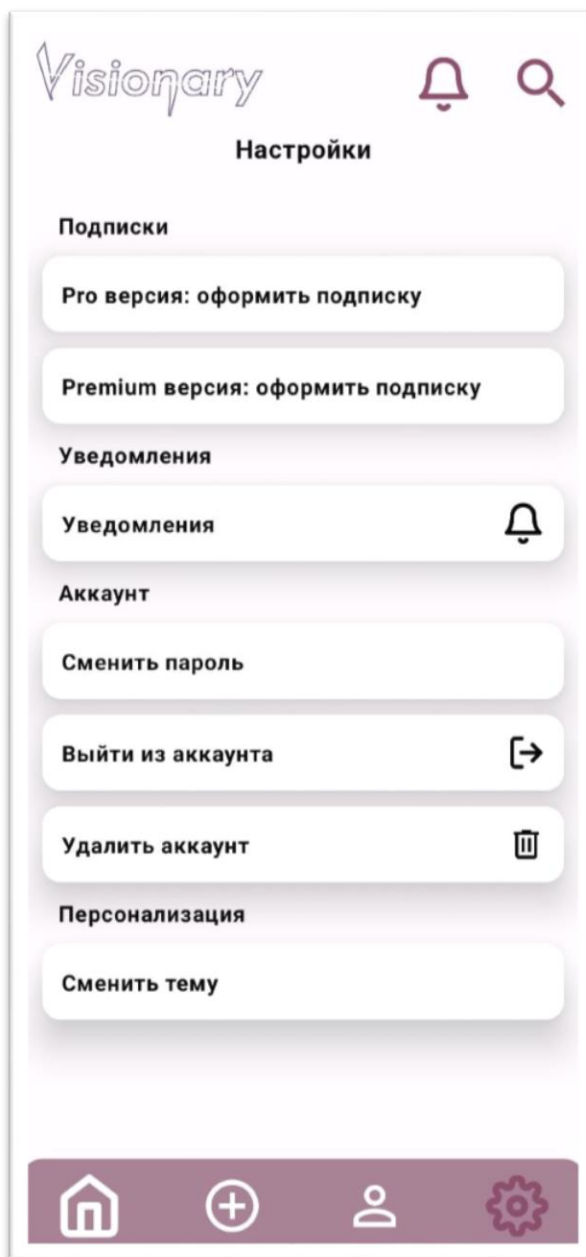


Рисунок 30 – Страница настроек приложения

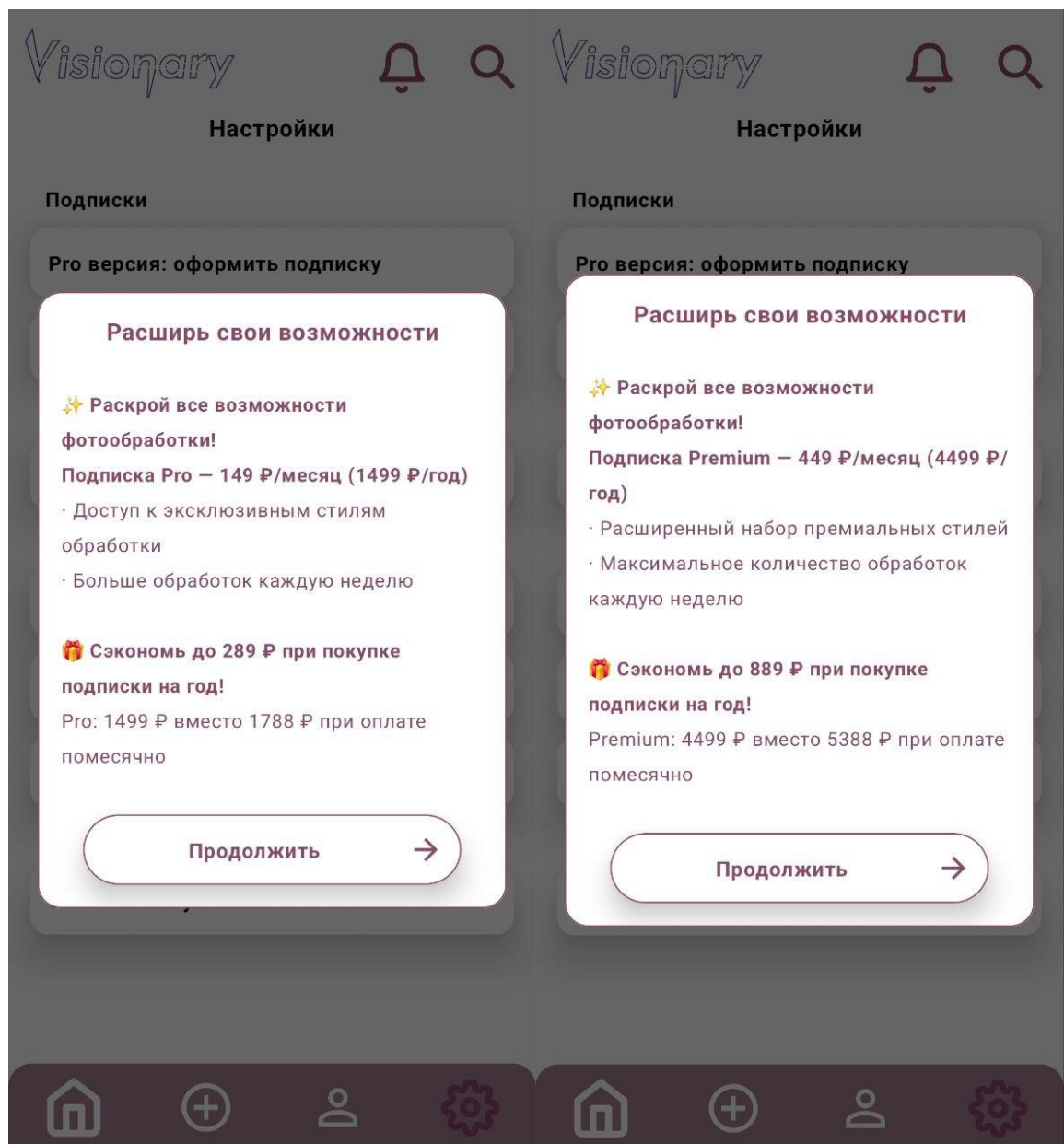


Рисунок 31 – Страницы информации для оформления подписки

4.7 Реализация интерфейса админ-панели

Админ-панель реализована для модерации деятельности пользователей основного приложения, работы с базой данных через удобный интерфейс, просмотра и работы с MinIO, а также просмотра метрик и угроз, связанных с нагрузкой на сервер, базу данных через Grafana, Prometheus и RabbitMQ. В админ-панели реализованы следующие возможности:

- доступ к базе данных PostgreSQL;
- просмотр документации по API приложения;

- доступ к базе MinIO;
- просмотр метрик в Grafana;
- просмотр метрик и предупреждений в Prometheus;
- просмотр и управление очередями сообщений в RabbitMQ.

4.7.1 Главная страница админ панели

Доступ ко всем возможностям осуществляется на главной странице админ-панели:

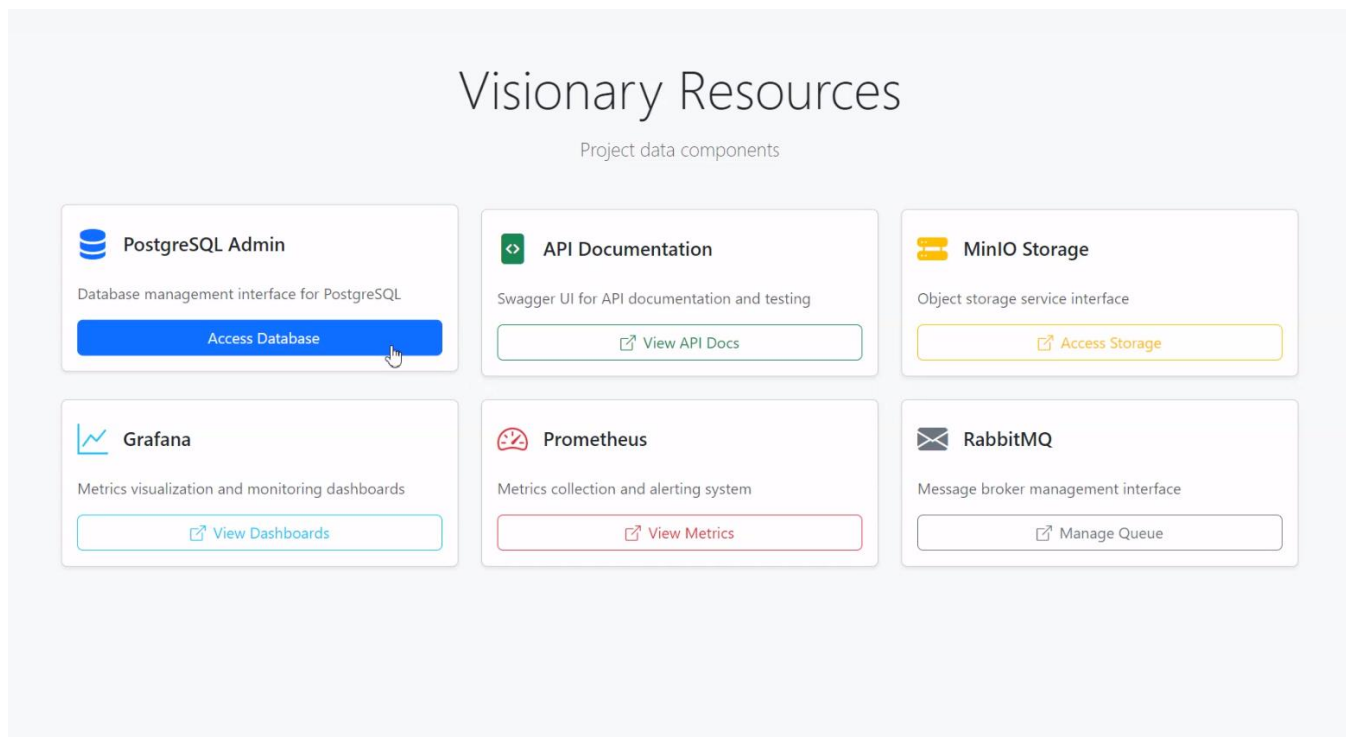
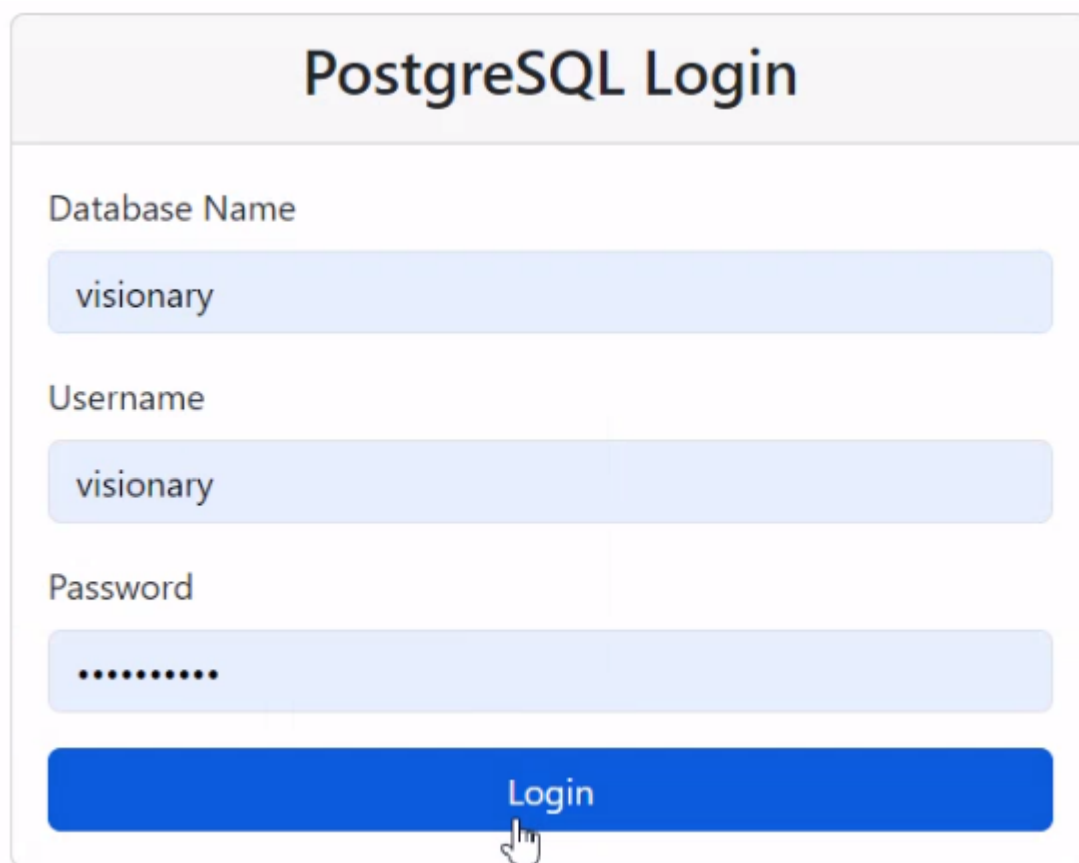


Рисунок 32 – Главная страница админ-панели

4.7.2 Работа с базой данных PostgreSQL

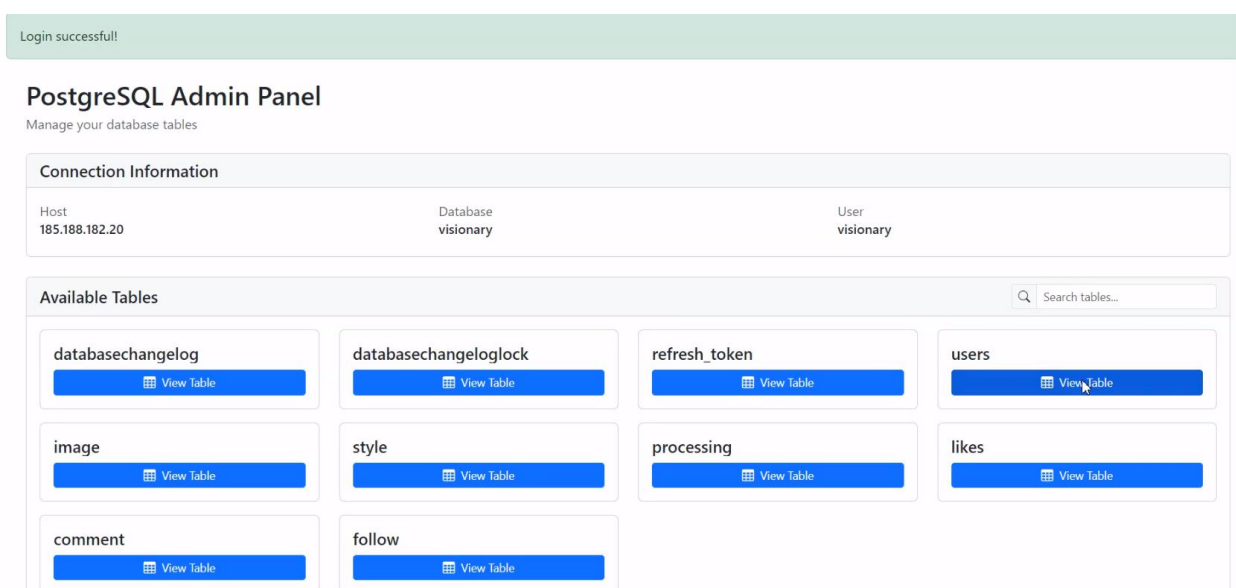
Вход на страницу осуществляется при помощи ввода имени базы, имени пользователя и пароля:



The image shows a login form titled "PostgreSQL Login". It contains three input fields: "Database Name" with the value "visionary", "Username" with the value "visionary", and "Password" with masked characters ".....". Below these fields is a blue "Login" button. A mouse cursor is pointing at the button.

Рисунок 33 – Вход на страницу для работы с PostgreSQL

После успешного ввода данных администратор попадает на страницу со всеми сущностями базы, помимо этого в верхней части пишется основная информация о подключении:



The image shows the "PostgreSQL Admin Panel" interface. At the top, a green banner displays "Login successful!". Below this, the title "PostgreSQL Admin Panel" is followed by the subtitle "Manage your database tables". A section titled "Connection Information" contains a table with the following data:

Connection Information		
Host 185.188.182.20	Database visionary	User visionary

Below the connection information is a section titled "Available Tables" with a search bar labeled "Search tables...". It displays a grid of table cards, each with a "View Table" button:

- databasechangelog
- databasechangeloglock
- refresh_token
- users
- image
- style
- processing
- likes
- comment
- follow

Рисунок 34 – Основная страница для работы с PostgreSQL

В каждую сущность можно зайти и увидеть все актуальные записи, а также выполнить удаление, сохранение и редактирование каждой записи:

PostgreSQL Admin visionary Logout

[Tables](#) / users

users

Showing up to 100 records

[← Back to Tables](#) [+ Add New Row](#)

Table Structure [Toggle](#)

Table Data

id	nickname	email	password	admin	description	avatar	created_at	active	Actions
4	Leon	sometingemail@gmail.com	\$2a\$10\$6NM0zqRm1kbj/O4hOOOSLu9O8fPkhXzjUkxdYtkASK6U8kaLre4jy	False	None	None	2025-04-27T21:10:04.493000	True	Edit Delete
5	MysteryLife	mystery@email.com	\$2a\$10\$MbYeiBoLCTIIBZyY8X7RYOY0QfwTn09iGKQkyl1UZ74GGSLQxKie	True	None	None	2025-04-28T00:10:00	True	Edit Delete
6	Pavel	kira@gmail.com	\$2a\$10\$5Zlr2WejycyxG.Chesq6s..nQq/1ylXvDg7zRbN6rDZ7NyqHgdKd2	False	None	None	2025-04-28T00:10:04.493666	True	Edit Delete
7	StarMan	ivan@gmail.com	\$2a\$10\$4J0ouplFc7MilclH8.qL/O9n2VuEYWwPOQtR8z/isslhtYYVSQzy	False	None	None	2025-04-28T00:10:04.493666	True	Edit Delete
8	Petr	petr@gmail.com	\$2a\$10\$QuKn7AqBM1QXMjwsWq5GPeQF0uuXSZ.W8LdXLT2dyfRlzaCDC509y	False	None	None	2025-04-28T00:10:04.493666	True	Edit Delete
9	Armin	mountain@gmail.com	\$2a\$10\$4nwl/Eu/8HiWqxUX8EMHBOyfCES9.riHL090EjUj1BOniBrZ6vMx.	False	None	None	2025-04-28T00:10:04.493666	True	Edit Delete
10	Nellmory	hello_world@gmail.com	\$2a\$10\$G4iPspMof3DRZd7kADpjusYkvR...Pavel.M...ACr0bC	False	None	None	2025-04-	True	Edit Delete

Рисунок 35 – Страница для работы с сущностью users из базы данных PostgreSQL

4.7.3 Просмотр документации по API приложения

Перейдя с основной страницы на документации, можно увидеть всю схему API для работы приложения с наборами методов (get, put, delete, post) для каждой сущности:

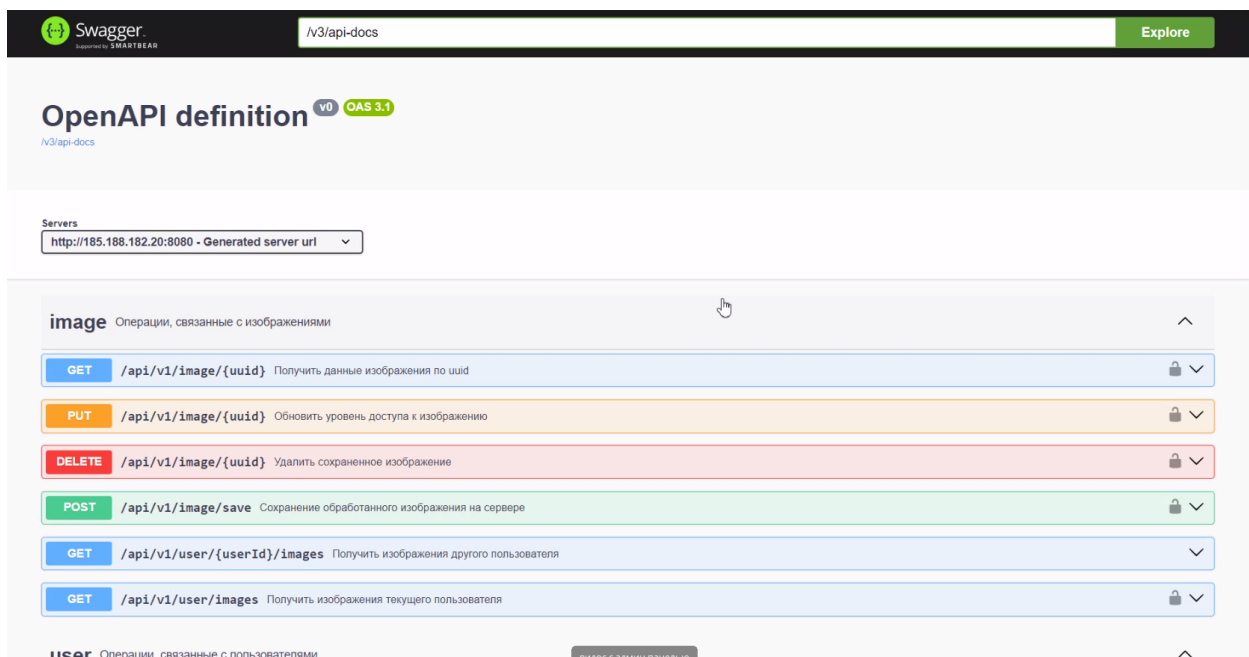


Рисунок 36 – Страница документации по API приложения

4.7.4 Страница доступа к базе MinIO

На этой странице можно увидеть все основные объекты этой базы – изображения пользователей, аватарки пользователей, работающие версии приложения:

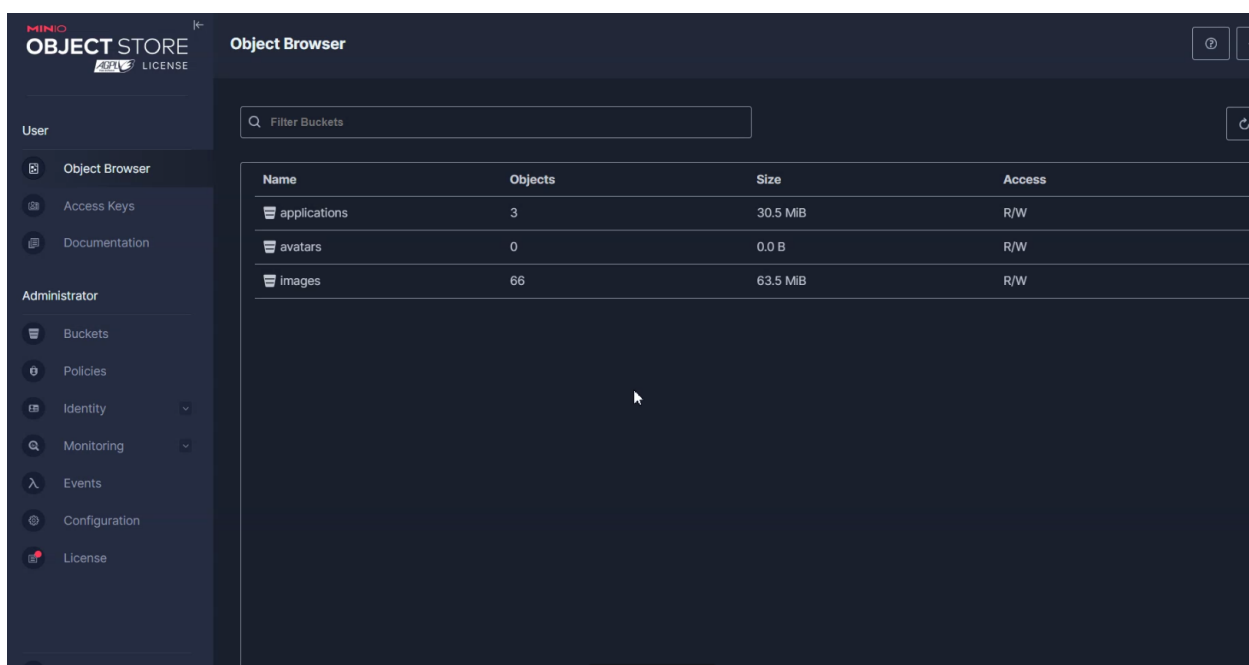


Рисунок 37 – Страница доступа к базе MinIO

4.7.5 Просмотр метрик при помощи Grafana

На данной странице можно увидеть данные собранные в метриках при помощи Prometheus в виде графиков и диаграмм:

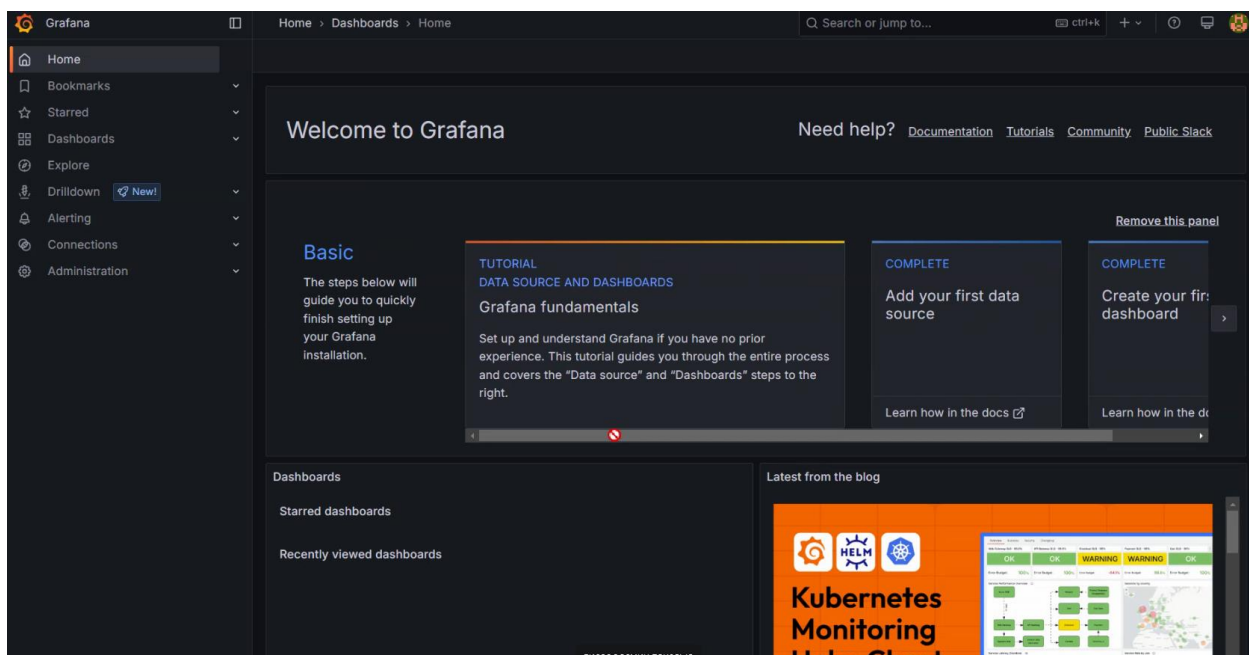


Рисунок 38 – Просмотр метрик при помощи Grafana

4.7.6 Просмотр метрик и предупреждений в Prometheus

На данной странице можно увидеть основные метрики приложения и предупреждения в работе, если имеются:

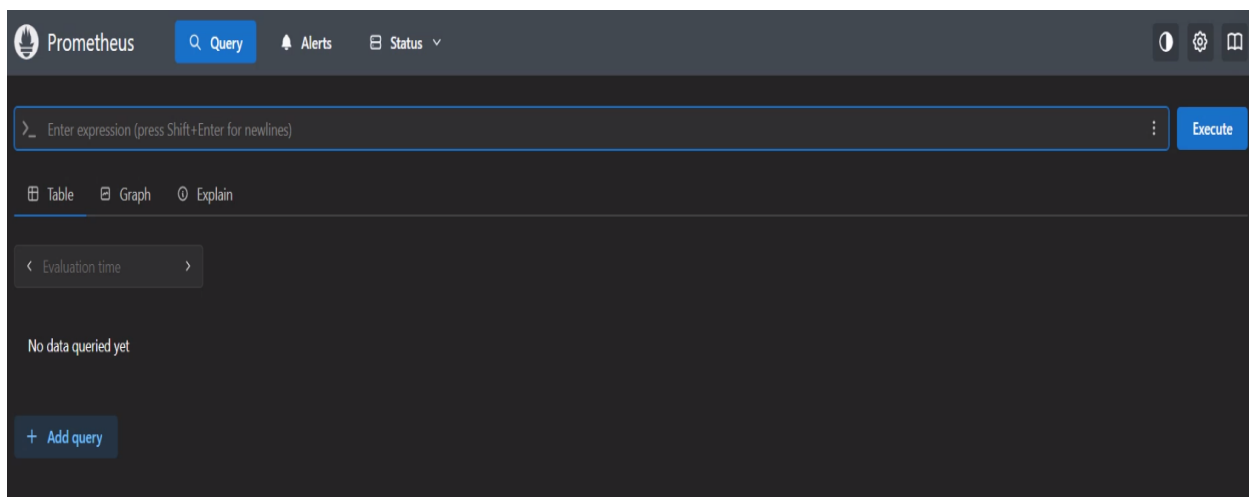


Рисунок 39 – Страница для просмотра метрик в Prometheus

4.7.7 Просмотр и управление очередями сообщений в RabbitMQ

На данной странице можно просматривать и управлять очередями сообщений приложения:

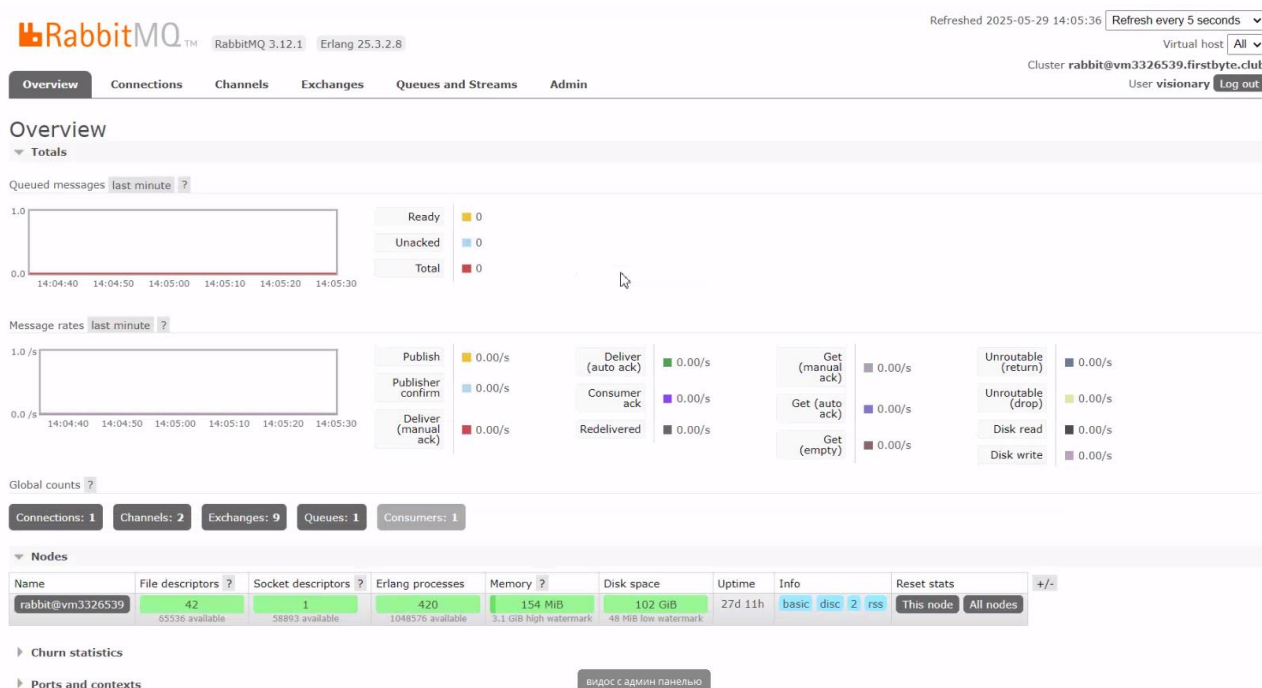


Рисунок 40 – Страница с работой над очередями в RabbitMQ

5 Тестирование

В рамках разработки мобильного приложения Visionary было проведено юнит-тестирование (unit testing) и ручное тестирование основных пользовательских сценариев. Целью тестирования было убедиться в корректной работе функциональных компонентов приложения, устойчивости серверной части и соответствии требованиям, заданным в техническом задании.

Unit-тесты были реализованы для проверки отдельных компонентов backend-приложения. Тестированию подверглись контроллеры, сервисы, репозитории, обработчики исключений и утилиты. Согласно результатам отчёта о покрытии кода, общее покрытие составило:

— Покрытие классов — 74%

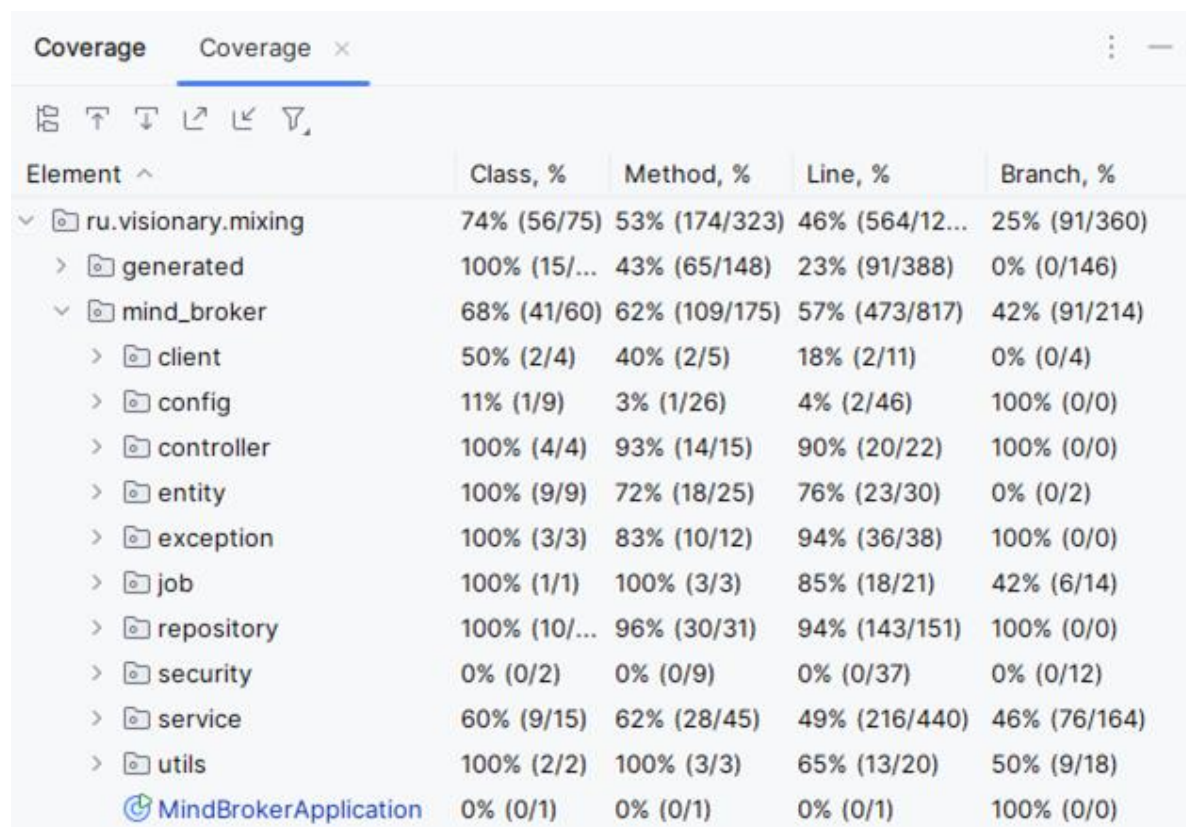
- Покрытие методов — 53%
- Покрытие строк кода — 46%
- Покрытие ветвлений — 25%

Наиболее полно протестированы следующие компоненты:

- контроллеры (controller) — 100% покрытие классов, 93% методов, 90% строк;
- репозитории (repository) — 100% классов и 96% методов;
- обработка исключений (exception) — 100% по всем метрикам;
- утилиты (utils) и фоновые задачи (job) — также полностью покрыты.

Менее покрыты модули service (60% классов, 49% строк). Тем не менее, общее покрытие достаточно для стабильной поддержки и масштабирования кода.

Скриншот отчёта покрытия приведён на рисунке 41.



Element ^	Class, %	Method, %	Line, %	Branch, %
ru.visionary.mixing	74% (56/75)	53% (174/323)	46% (564/12...)	25% (91/360)
> generated	100% (15/...	43% (65/148)	23% (91/388)	0% (0/146)
> mind_broker	68% (41/60)	62% (109/175)	57% (473/817)	42% (91/214)
> client	50% (2/4)	40% (2/5)	18% (2/11)	0% (0/4)
> config	11% (1/9)	3% (1/26)	4% (2/46)	100% (0/0)
> controller	100% (4/4)	93% (14/15)	90% (20/22)	100% (0/0)
> entity	100% (9/9)	72% (18/25)	76% (23/30)	0% (0/2)
> exception	100% (3/3)	83% (10/12)	94% (36/38)	100% (0/0)
> job	100% (1/1)	100% (3/3)	85% (18/21)	42% (6/14)
> repository	100% (10/...	96% (30/31)	94% (143/151)	100% (0/0)
> security	0% (0/2)	0% (0/9)	0% (0/37)	0% (0/12)
> service	60% (9/15)	62% (28/45)	49% (216/440)	46% (76/164)
> utils	100% (2/2)	100% (3/3)	65% (13/20)	50% (9/18)
MindBrokerApplication	0% (0/1)	0% (0/1)	0% (0/1)	100% (0/0)

Рисунок 41 – Покрытие кода unit-тестами в проекте Visionary

По итогам тестирования можно сделать вывод, что приложение Visionary демонстрирует высокую стабильность и функциональную полноту. Реализованные юнит-тесты покрывают основную бизнес-логику проекта, а ручное тестирование подтверждает корректность поведения всех ключевых пользовательских сценариев. Это создаёт уверенную основу для дальнейшего расширения функционала и масштабирования системы.

6 Аналитика

В качестве системы для сбора статистики работы приложения был выбран сервис Amplitude. В качестве достоинств данного сервиса выделяют: – простота интеграции с мобильными приложениями; – наглядная визуализация пользовательских путей; – возможность отслеживания событий в реальном времени.

6.1 Воронки

Для формирования аналитики по приложению Visionary были созданы четыре воронки: – регистрация; – стилизация фото; – социальные взаимодействия; – использование премиум-функций.

В таблицах 2-5 представлены этапы каждой из четырех воронок:

Таблица 2 – Процент конверсии регистрации

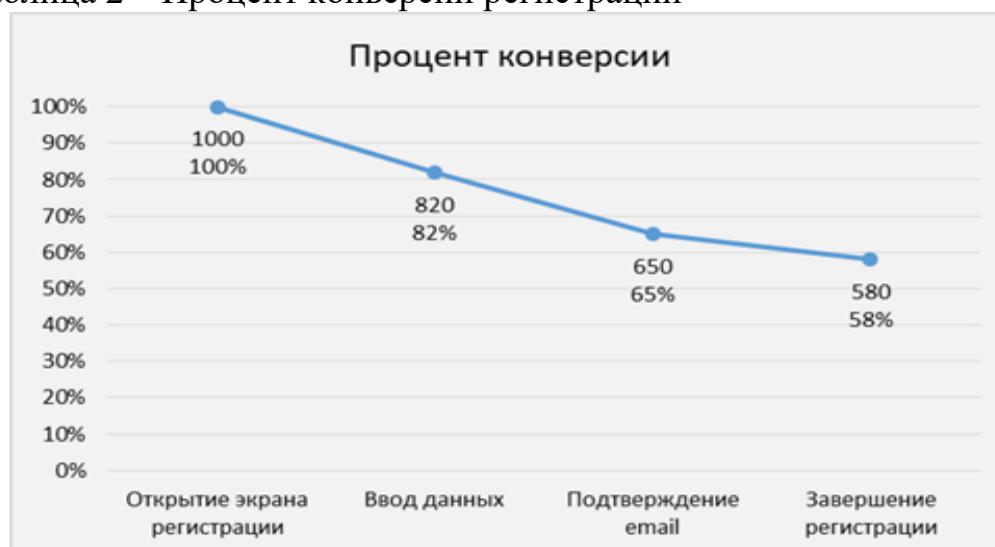


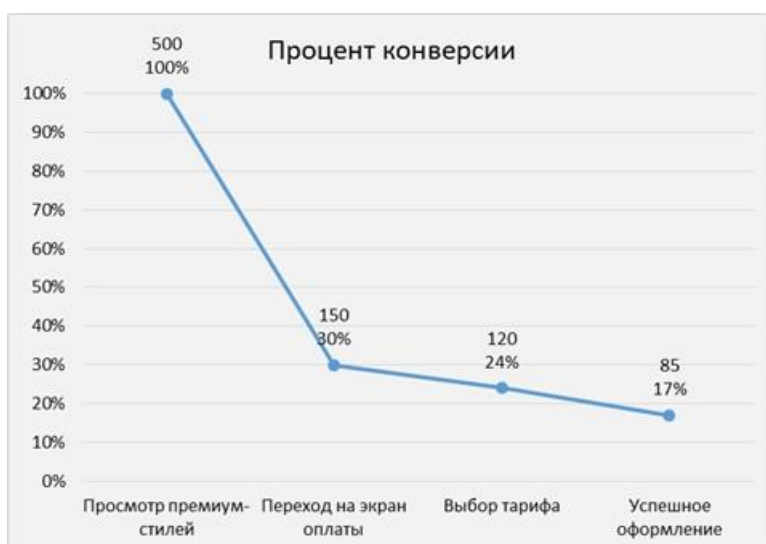
Таблица 3 – Процент конверсии стилизации фото



Таблица 4 – Процент конверсии социального взаимодействия



Таблица 5 – Процент конверсии использования премиум-функций



Анализ воронки регистрации показал, что 58% пользователей успешно завершают процесс, что является хорошим показателем для мобильного приложения. Наибольший отток наблюдается на этапе подтверждения email.

Воронка стилизации фото демонстрирует высокую вовлеченность пользователей в основной функционал приложения, с конверсией 72% до этапа сохранения результата. Однако только 42,7% пользователей публикуют обработанные фото в ленте, что указывает на необходимость стимулирования социальной активности.

6.2 Статистика

На рисунке 42 продемонстрирована статистика по переходам на различные экраны приложения. Наибольшей популярностью пользуется главный экран (37,0%) и экран ИИ-обработки (22,2%), что подтверждает правильность фокуса на основной функциональности приложения. Кнопка "Лайк" с показателем 14,8% свидетельствует о хорошей социальной вовлеченности пользователей.



Рисунок 42 – Статистика по переходам на экраны приложения

В таблице 5 представлена аналитика воронки конверсии для премиум-функций, которая показывает, что 17% пользователей, заинтересовавшихся платными стилями, совершают покупку. Этот показатель превышает среднерыночный (10-15%) для приложений с моделью монетизации freemium.

Собранные метрики позволяют выявить ключевые точки взаимодействия пользователей с приложением и определить направления для дальнейшей оптимизации пользовательского опыта и повышения конверсии.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы были последовательно решены все задачи, поставленные в техническом задании Visionary. Проведён анализ предметной области, изучены и реализованы современные нейросетевые подходы к переносу стиля; исследованы существующие аналоги на русском и зарубежном рынках, что позволило выявить вектор развития; спроектирована трёхуровневая архитектура клиент-серверного приложения и разработан ненагруженный интерфейс для удобства пользователей.

В результате было реализовано мобильное приложение Visionary, включающее следующие ключевые модули:

- авторизация и регистрация пользователя (через e-mail) с использованием JWT-токенов;
- загрузка изображений из галереи и с камеры мобильного устройства;
- применение фильтров к фотографиям перед или после стилизации (яркость, контрастность);
- модуль нейросетевого переноса стиля с предустановленными и платными стилями («киберпанк», «аниме», «нуар», «ван-Гог» и другие);
- сохранение и экспорт обработанных изображений в галерею и сторонние приложения;
- публикация изображений в ленте приложений (личное/публичное), просмотр ленты, поиск и подписка на авторов;
- взаимодействие с контентом: лайки, комментарии;
- управление профилем: редактирование данных, настройка приватности и тем оформления;

Приложение успешно прошло тестирование на устройствах с Android 8.0 и выше, продемонстрировало приемлемую производительность (время обработки фото до 5 минут, отклик сервера не более 15 секунд) и устойчивость

к наиболее распространённым видам атак (защита от SQL-инъекций, шифрование персональных данных). Минималистичный интерфейс и адаптивная верстка обеспечивают комфортное взаимодействие на экранах различных размеров, а поддержка тёмной темы удовлетворяет современным требованиям UX.

Полученный результат соответствует целям и задачам курсовой работы, его практическая ценность заключается в доступности инструмента для быстрого и качественного создания креативного контента на мобильных устройствах. В перспективе возможна интеграция дополнительных алгоритмов комбинированных стилей, внедрение системы рекомендаций на основе поведения пользователей и расширение социального функционала (личные сообщения, групповые чаты), что позволит увеличить вовлечённость аудитории и развитие сообщества Visionary.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Вигерс, К. Разработка требований к программному обеспечению / К. Вигерс, Дж. Битти. – 3-е изд., дополненное. – Москва : Русская редакция, 2014. – 736 с. – ISBN 978-5-7502-0433-5. – Текст : непосредственный.
2. Официальная документация Python [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/> (дата обращения: 12.06.2025). – Загл. с экрана.
3. Официальная документация Spring Framework [Электронный ресурс]. – Режим доступа: <https://spring.io/projects/spring-framework> (дата обращения: 12.06.2025). – Загл. с экрана.